

Jagiellonian University
Department of Theoretical Computer Science

Aliaksandr Skvarniuk

Implementation and comparison of Lempel–Ziv factorization algorithms

Bachelor's Thesis

Supervisor: dr hab. Krzysztof Turowski

Cracow, 2026

Contents

1	Introduction	2
1.1	Research topic	2
1.2	Definitions	4
1.2.1	Strings	4
1.2.2	The factorization problem	4
1.2.3	Index structures	5
1.2.4	LZ77 common definitions	7
1.2.5	LZ78 common definitions	8
1.2.6	Compression ratio	10
2	LZ77	11
2.1	Idea	11
2.1.1	Theoretical vs practical approach	11
2.1.2	Pseudocode	13
2.1.3	Additional notes	15
2.2	Correctness	15
2.2.1	The parsing is well defined	16
2.2.2	The decoder reconstructs the source	16
2.3	Complexity	16
3	LZ78	17
3.1	Idea	17
3.1.1	Pseudocode	18
3.2	Correctness	20
3.2.1	The parsing is well defined	21
3.2.2	The decoder reconstructs the source	21
3.3	Complexity	21
4	LZ77: Chen–Puglisi–Smyth variant	23
4.1	Idea	23
4.1.1	From online to offline	23
4.1.2	<i>CPS1a</i> approach description	23
4.1.3	Pseudocode	25
4.1.4	Encoder and decoder of <i>CPS1a</i> version	26
4.2	Correctness	27
4.3	Complexity	28
5	LZ77: Kärkkäinen–Kempa–Puglisi variant	29
5.1	Idea	29
5.1.1	KKP3 approach description	29

5.1.2	Pseudocode	30
5.2	Correctness	31
5.2.1	Two candidates suffice	32
5.3	Complexity	33
6	Comparison and Experiments	34
6.1	Comparison	34
6.1.1	Repository	34
6.1.2	Approaches	34
6.1.3	Time and Space	34
6.2	Experiments	35
6.2.1	Runs	35
6.2.2	Results summary	41
7	Future implementations	43
7.1	<i>CPS</i> algorithms	43
7.1.1	<i>CPS1b</i> and <i>CPS1c</i>	43
7.1.2	<i>CPS2</i>	43
7.1.3	<i>CPS3</i>	44
7.2	<i>KKP</i> algorithms	44
7.2.1	<i>KKP2s</i>	44
7.2.2	<i>KKP2n</i>	45

Large language models were used as an aid in literature searches and English spelling checks.

Chapter 1

Introduction

1.1 Research topic

In this work we are going to take a look on the original versions as well as modern improvements and applications of Lempel-Ziv LZ77 and LZ78 compression methods.

LZ77 and LZ78, which are widely used in text compression since the late 70's, have recently attracted more attention due to modern research in data compression with a specific purpose based on these methods, namely in the directions of factorization algorithms ([1, 2, 3, 4, 5]), run-length encoding (RLE) ([6]), self-indexing ([7, 8]), grammar-compression ([9]), algorithms focused on encoding of the factors (LZSS, LZW) ([10, 11]).

We take a closer look at LZ77 and LZ78 versions with different factorization algorithms in this work, so below are works related to our focus.

Year	Authors	Ref	Time	Space
1977	Ziv & Lempel	[1]	$\mathcal{O}(n \cdot w)$	$\mathcal{O}(w)$
1989	Fiala & Greene	[12]	$\mathcal{O}(n)$	$\mathcal{O}(w)$
2008	Chen, Puglisi & Smyth	[13]	$\Theta(n)$	$\mathcal{O}(n)$
2012	Kärkkäinen, Kempa & Puglisi	[3]	$\Theta(n)$	$\mathcal{O}(n)$
2017	Fischer et al.	[4]	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Table 1.1: Algorithms for the LZ77 factorization. Here n is the length of the string and w the length of the window, when the method uses one.

Year	Authors	Ref	Time	Space
1978	Ziv & Lempel	[2]	$\mathcal{O}(n)$ expected	$\mathcal{O}(w)$
2015	Nakashima et al.	[14]	$\Theta(n)$	$\mathcal{O}(n)$
2017	Fischer et al.	[4]	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Table 1.2: Algorithms for the LZ78 factorization. Here n is the length of the string and w the length of the block.

In this work we are going to have a look on problem of string compression in perspective of how can we use repetitions of blocks of symbols inside string for this task. In all approaches

covered by this work we are going to use knowledge about already processed data in order to represent non-processed one of the same source. Intuition behind this is such that, for example, if we have string *abaababa*, it would be nice to represent *aba* part on the end of the string with, again, for example, pair $(p = 1, l = 3)$, where p is some pointer to the start of the string and l is the length of repeated fragment, noteworthy, they might even overlap, depending on the approach we dive into, for example for same *abaababa* we take last two occurrences of *aba* — it is easily seen that they overlap. As we can see, the substring that was already seen can help us replace future substrings with some shorter data that indicates the event of their appearance, what in terms of compression means potentially shorter representation of these future substrings. In order to get such representation we are going to work with factorization of the string. The factorization approach in this work won't be simple division into blocks, but rather the one that follows intuition from above.

Now, let us introduce two definitions that are needed to understand the whole framework. Note, the distinction of next two definitions is ours, they refer only to the content of this work, and not characterize corresponding classes of problems, but rather the side from which we look at these problems here.

Definition 1.1 (Offline factorization). *In the offline setting the whole source text is available for computation of factorization. Each factor may point anywhere to the left where it was previously seen.*

Definition 1.2 (Online factorization). *In the online setting the source is read left to right in order to compute factorization, but having in working memory only fixed length of previously seen string, not the whole processed one. A currently formed factor may therefore point only inside part which lies in working memory at this moment.*

The *online* setting allows us to produce factors, regardless of data that is no longer in working memory, which gives us a way to deal with dynamically arriving data and even hard data, which is too big to fully hold in working memory. The two settings differ in application in how much of the source a factor may point back into, but are not fully separate, as we can set working memory of *online* on par with the size of the source and work with them as with *offline* ones.

In this work we implement and compare four algorithms: the original sliding-window *LZ77* [1], *LZ78* with the incremental parsing of [2], the *CPS1a* variant of *LZ77* of [13], and the variant of *LZ77* from [3], referred to as *KKP3*. The concepts that are not clear in previous sentence we will try to define and explain in the next section. The applications of settings mentioned above are seen in our work through design of compression models. Each model consists of encoder and decoder, and factorization setting is applied inside encoder as follows: [1, 2] designed in the online setting, [13, 3] in the offline one.

The rest of the work is organized as follows. Section 1.2 collects the definitions used throughout and their explanations. Chapters 2 to 5 describe the four algorithms, each in the same order: the idea, the correctness, the complexity. Chapter 6 compares them and reports the measurements of experiments.

1.2 Definitions

1.2.1 Strings

Throughout this work, let $\mathcal{A}$ denote a finite ordered alphabet of size $\alpha = |\mathcal{A}|$.

Once we have the source string it is useful to have a special symbol called *sentinel*, which is not from the source's alphabet, but is smaller than the ones from $\mathcal{A}$. For example, the addition of *sentinel* at the end of the string makes the order on its suffixes strict, since no suffix can then be a prefix of another.

Definition 1.3 (Sentinel). *A sentinel is some letter $\$ \notin \mathcal{A}$, smaller than every letter of alphabet $\mathcal{A}$.*

1.2.2 The factorization problem

Every scheme in this work cuts the source string into consecutive factors and replaces each factor by a reference to already seen part, we call it factorization. Let us first introduce the factorization of a whole string, that the algorithm has to compute.

Definition 1.4 (LZ factorization, [1]). *The LZ factorization of x is the decomposition $x = w_1 w_2 \dots w_k$ into nonempty factors such that each w_j satisfies one of conditions below:*

- (a) *is a letter that does not occur in $w_1 w_2 \dots w_{j-1}$,*
- (b) *is the longest substring that occurs at least twice in $w_1 w_2 \dots w_j$.*

Example 1.1 (LZ-factorization of string *abaababa*). *Let $x = abaababa$. Its LZ factorization is*

$$x = a \cdot b \cdot a \cdot aba \cdot ba,$$

where $w_1 = a$ and $w_2 = b$ are new letters, $w_3 = a$ is the longest substring occurring twice in aba , $w_4 = aba$ occurs again at position 1, and $w_5 = ba$ occurs again at position 2.

In later implementations we are going to read Definition 1.4 left to right and repeatedly ask the same question in order to form next factor: starting here, how long a factor has already been seen and where, that's why we also need to define such answer, which is pair (p_i, l_i) :

Definition 1.5 (Longest previous factor, [3]). *The longest previous factor at a position $i \in [1 \dots n]$ is a pair (p_i, l_i) with $p_i < i$ and*

$$x[p_i \dots p_i + l_i - 1] = x[i \dots i + l_i - 1],$$

and $l_i > 0$ is as large as possible.

Put differently, $x[i \dots i + l_i - 1]$ is the longest prefix of suffix i that also starts at some earlier position p_i . If $x[i]$ is the leftmost occurrence of its letter, no such pair exists, and one sets $p_i = x[i]$ and $l_i = 0$. There may be several fitting p_i and it does not matter which of them is taken. Longest previous factor array may be computed in $\mathcal{O}(n)$ time (see [15]).

As extension of previous definition, we need to mention that the factorization can be seen as array of pairs or two arrays that answer the question from before for each position of source string.

Definition 1.6 (POS and LEN arrays, [13]). For a position $i \in [1, \dots, n]$, $\text{LEN}[i]$ is the length of the longest factor starting at i that has an earlier occurrence in x , and $\text{POS}[i]$ is the starting position of one such occurrence, so that

$$x[\text{POS}[i] \dots \text{POS}[i] + \text{LEN}[i] - 1] = x[i \dots i + \text{LEN}[i] - 1], \quad \text{POS}[i] < i.$$

$\text{LEN}[i] = 0$ and $\text{POS}[i] = i$ when no such occurrence exists.

For visual example we can look at string *abaababa*. Both arrays POS and LEN can be filled by hand straight from the description above:

i	1	2	3	4	5	6	7	8
$x[i]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>
$\text{LEN}[i]$	0	0	1	3	2	3	2	1
$\text{POS}[i]$	1	2	1	1	2	1	2	1

Table 1.3: POS and LEN for $x = \textit{abaababa}$.

At $i = 1$ and $i = 2$ nothing has been seen yet, so $\text{LEN} = 0$ and the pointer is unused. At $i = 4$ the factor *aba* repeats the prefix $x[1 \dots 3]$, and at $i = 6$ it repeats it again. The pointers listed above happen to be the leftmost occurrences, but the algorithm does not require this, and it may report any earlier one. In this string no factor overlaps its own earlier occurrence, but in general it may: for $x = \textit{aaaa}$ we get $\text{POS}[2] = 1$ and $\text{LEN}[2] = 3$, so the factor $x[2 \dots 4]$ is read from $x[1 \dots 3]$, a range it shares three positions with.

1.2.3 Index structures

Computing (p_i, l_i) position by position and comparison by comparison is quadratic. Offline algorithms implemented by us avoid that by sorting the suffixes first: once they are in lexicographical order, suffixes sharing a long prefix stand next to each other, and a repeated factor becomes a contiguous block instead of a search.

Definition 1.7 (Suffix array (SA), [16]). The Suffix Array of x is an array $[1 \dots n]$ in which $\text{SA}[j] = i$ iff suffix i is the j -th in lexicographical order among all the suffixes of x .

Year	Authors	Ref	Running time
1972	Karp, Miller & Rosenberg	[17]	$\mathcal{O}(n \log n)$
2003	Kärkkäinen & Sanders	[18]	$\mathcal{O}(n)$
2003	Ko & Aluru	[19]	$\mathcal{O}(n)$
2007	Larsson & Sadakane	[20]	$\mathcal{O}(n \log n)$
2009	Nong, Zhang & Chan	[16]	$\mathcal{O}(n)$

Table 1.4: In [21] a handful of algorithms can be found for building suffix array

The structure that holds for each pair of suffixes adjacent in SA, the length of their common prefix is desirable, since suffix array says in what order the suffixes stand, but not

how much two neighbours have in common, what turns adjacency in SA into information about repeats of similar prefixes, what in its turn will boil down to the fact that desirable factors are among them. Suffix arrays may be computed in linear time, see Table 1.4.

Definition 1.8 (Longest Common Prefix Array (LCP), [15]). *Let $\text{lcp}(i_1, i_2)$ denote the length of the longest common prefix of suffixes i_1 and i_2 . The LCP array stores the longest common prefixes of suffixes adjacent in SA,*

$$\text{LCP}[j] = \text{lcp}(\text{SA}[j-1], \text{SA}[j]), \quad j \in [2, 3, \dots, n]$$

with the sentinel values $\text{LCP}[1] = \text{LCP}[n+1] = -1$.

Given x and its suffix array, the LCP array can be computed in $\Theta(n)$ time and $\mathcal{O}(n)$ additional space ([15]), so having SA it costs one linear pass and one more integer array. Example of SA and LCP arrays of string *abaababa*:

j	1	2	3	4	5	6	7	8	9
$x[j]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	
$\text{SA}[j]$	8	3	6	1	4	7	2	5	
$\text{LCP}[j]$	-1	1	1	3	3	0	2	2	-1

Table 1.5: SA and LCP for $x = \textit{abaababa}$.

A repeated substring shows up as a block of neighbouring entries of SA whose LCP are not decreasing, so they share same prefix, and this definition names such a block.

Definition 1.9 (Group, [13]). *For $l \geq 1$, a group of level l is a maximal range $[a \dots b]$ of SA with $a < b$ such that $\text{LCP}[j] \geq l$ for all $j \in [a+1 \dots b]$, while $\text{LCP}[a] < l$ and $\text{LCP}[b+1] < l$. All suffixes $\text{SA}[a], \dots, \text{SA}[b]$ then share a prefix of length l .*

The groups of Definition 1.9 for the same string *abaababa*, read off Table 1.5, are as follows:

range	level	suffixes	common prefix
$[1 \dots 5]$	1	8, 3, 6, 1, 4	<i>a</i>
$[3 \dots 5]$	2, 3	6, 1, 4	<i>ab, aba</i>
$[6 \dots 8]$	1, 2	7, 2, 5	<i>b, ba</i>

Table 1.6: Groups of $x = \textit{abaababa}$.

Note that $[3 \dots 5]$ lies inside $[1 \dots 5]$, and that one range can be a group at several levels at once, where level is length of longest common prefix of these suffixes.

A group gathers candidates for p_i , but it can be large, and scanning it may be costly. The next pair of arrays narrows the search to just two entries. The intuition is that the closer a suffix stands to suffix i in SA the longer the prefix it shares with it, so it is enough to look at the nearest neighbour on each side that starts earlier in the text.

Definition 1.10 (Inverse suffix array (ISA), [3]). *The inverse suffix array ISA is the inverse permutation of SA: $\text{ISA}[i] = j$ iff $\text{SA}[j] = i$, that is, $\text{ISA}[i]$ is the position of suffix i in SA.*

Definition 1.11 (Next and previous smaller values (NSV and PSV), [3]). *For a position $i \in 1 \dots n$ of S ,*

$$\begin{aligned}\text{NSV}_{\text{lex}}[i] &= \min\{j \in [i+1 \dots n] \mid \text{SA}[j] < \text{SA}[i]\}, \\ \text{PSV}_{\text{lex}}[i] &= \max\{j \in [1 \dots i-1] \mid \text{SA}[j] < \text{SA}[i]\},\end{aligned}$$

with the value 0 when the set is empty. The corresponding positions of the text are

$$\begin{aligned}\text{NSV}[i] &= \text{SA}[\text{NSV}_{\text{lex}}[\text{ISA}[i]]], \\ \text{PSV}[i] &= \text{SA}[\text{PSV}_{\text{lex}}[\text{ISA}[i]]],\end{aligned}$$

where we again assume the value 0 when the respective index equals 0.

Example 1.2. *NSV and PSV.] For $x = \text{abaababa}$ with $\text{SA} = [8, 3, 6, 1, 4, 7, 2, 5]$ of Table 1.5:*

i	1	2	3	4	5	6	7	8
$\text{SA}[i]$	8	3	6	1	4	7	2	5
$\text{ISA}[i]$	4	7	2	5	8	3	6	1
$\text{PSV}[i]$	0	1	0	1	2	3	4	0
$\text{NSV}[i]$	0	0	1	2	0	1	2	3

Take, for example, $i = 6$. Suffix 6 sits at $\text{ISA}[6] = 3$ in SA, and the neighbouring entries are $\text{SA}[2] = 3$ on the left and $\text{SA}[4] = 1$ on the right. Both are smaller than 6, so they are the nearest such on each side and $\text{PSV}[6] = 3$, $\text{NSV}[6] = 1$.

However, for $i = 1$ both values are 0: suffix 1 sits at $\text{ISA}[1] = 4$, and no entry of SA on either side of it holds a value smaller than 1, as there is none. The same happens at $i = 3$ on the left and at $i = 5$ on the right.

The structures of this section offer two ways of arriving at the factorization, and they correspond to the two offline algorithms of this work. One fills the arrays of Definition 1.6 completely, walking along SA and LCP and letting every group of Definition 1.9 assign its pointers as it closes ([13]). Another computes the pair of Definition 1.5 only at the positions where a factor actually starts, choosing between the two candidates ([3]). The first computes all n entries and reads k of them, the second computes only k as the factorization proceeds.

1.2.4 LZ77 common definitions

The online setting of Definition 1.2 has to be made concrete: something must say how much of the already read source the encoder is allowed to keep. In [1] that role is played by a buffer of fixed length, which slides along the source as factors are emitted.

Definition 1.12 (Sliding window, [1]). *The encoder keeps a buffer B_i of n letters. Its last L_s positions hold the part of the source that has been read but not yet factored; the first $n - L_s$ positions form the sliding window, the history against which the next word is matched. Once a word of length l_i has been encoded, the buffer is shifted left by l_i positions and refilled with the next l_i letters of the source:*

$$B_{i+1} = B_i(l_i + 1, n) S(h_i + 1, h_i + l_i),$$

where h_i is the position of S occupied by the last letter of B_i .

Definition 1.13 (Maximal word length (L_s), [1]). L_s is the prescribed upper bound on the length of a source word: $1 \leq l_i \leq L_s$ for every i . It bounds the encoding delay, since the encoder never waits for more than L_s letters before emitting a codeword, and it fixes the width of the length field of the codeword.

The window bounds how far back a factor may point to. A second parameter (L_s) bounds how long it may be, since only fixed amount of last symbols in window are still not processed, they either preserved from last iterations or just were added from source. The two together are what make a codeword of fixed width possible.

Definition 1.14 (Codeword length (L_c), [1]). Every source word S_i is mapped to a codeword $C_i = C_{i1}C_{i2}C_{i3}$ over the same alphabet $\mathcal{A}$, where C_{i1} is the radix- α representation of the pointer $p_i - 1$, C_{i2} that of the length $l_i - 1$, and C_{i3} is the last letter of S_i .

All codewords share the fixed length

$$L_c = 1 + \lceil \log_\alpha(n - L_s) \rceil + \lceil \log_\alpha L_s \rceil$$

The online counterpart of Definition 1.4, stated for the prefix the encoder currently holds instead of the whole source, also fit to bounded on both sides of currently available text in working memory. In algorithm [1], since we work on sliding window B_{now} (see Definition 1.12), the string S from below is in fact the current buffer B_{now} .

The rule below is an operation on a string: given a prefix of it, find the longest continuation that starts at some position inside that prefix. The pointer must lie inside the prefix, but the match itself is allowed to run past its end and overlap the continuation being read, so the two ranges may share positions. Such continuation is the next factor of string representation in factorization process with allowed overlapping.

Definition 1.15 (Reproducible extension, [1]). Let $S(1, j)$ be a proper prefix of a string S and let $i \leq j$. Denote by $L(i)$ the largest nonnegative integer $l \leq |S| - j$ such that

$$S(i, i + l - 1) = S(j + 1, j + l),$$

and let p be a position of $S(1, j)$ for which $L(p) = \max_{1 \leq i \leq j} L(i)$. The substring $S(j + 1, j + L(p))$ is called the reproducible extension of $S(1, j)$ into S , and p is called the pointer of the reproduction.

1.2.5 LZ78 common definitions

LZ78 introduces an alternative referencing scheme. A factor is no longer a substring addressed by position and length, but an entry of a dictionary built dynamically and working on distinct entries. The parsing is therefore arranged so that the entries come out distinct by construction.

Definition 1.16 (Incremental parsing, [2]). A parsing $x = y_1 y_2 \cdots y_p$ is incremental if the phrases $y_1, \dots, y_{p-1}$ are pairwise distinct and, for every j , the word obtained from y_j by deleting its last letter is either empty or equal to some earlier phrase y_i with $i < j$. Writing $\pi(j) = i$ for that index and a_j for the deleted letter, every phrase satisfies $y_j = y_{\pi(j)} a_j$ with $\pi(j) < j$, where the index 0 denotes the empty word. The last phrase y_p need not be distinct from the earlier ones.

Example 1.3 (Incremental parsing of *abaababa*). Let $x = abaababa$ with the block length $n = 8$, then the parsing is

$$x = a \cdot b \cdot aa \cdot ba \cdot ba$$

In fact it is easier to follow, once displayed as *trie*, what can be seen below next definition.

Since every phrase extends an earlier one by one letter, the phrases nest as a tree, and finding the next one becomes a walk on it from root down by already met substrings.

Definition 1.17 (Dictionary trie, [2]). A rooted tree whose edges are labeled by letters of $\mathcal{A}$, so that the word spelled out on the path from the root to a node is exactly one phrase, and every phrase corresponds to exactly one node. The root is numbered 0 and spells the empty word. The node of the j -th phrase is numbered j .

Let π be a function such that $\pi(j)$ is the index in their order of creation and refers to the parent of the node j in trie: $y_j = y_{\pi(j)}a_j$, the node j is a child of the node $\pi(j)$ along the edge labeled a_j , new phrase is inserted as a single new leaf when it emerges.

Where $\pi(j)$ is notation for operation taken from [2], and it gives the parent node of j . Extending what is described before this definition, we have that the phrases of Definition 1.16 are stored in a *trie* (see example in Figure 1.1).

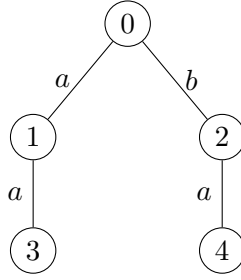


Figure 1.1: The corresponding trie for $x = abaababa$, with nodes numbered as they emerge.

The path from the root to node j spells the phrase y_j : $y_1 = a$, $y_2 = b$, $y_3 = aa$, $y_4 = ba$.

Since we are going to mention in corresponding chapter the theoretical reasoning behind construction, that the authors provide in LZ78 work ([2]), we need to add needed definitions, as the compression model here is not stated against other compressors in general but against a fixed class of machines.

Definition 1.18 (Finite-state encoder, [2]). A finite-state encoder is a quintuple $E = (Z, A, B, g, f)$, where Z is a finite set of states, A is the input alphabet, B is a finite set of output words over a binary output alphabet, $g : Z \times A \rightarrow Z$ is the next-state function and $f : Z \times A \rightarrow B$ is the output function. Fed with $x = x_1x_2\cdots$, the encoder emits $y_i = f(z_i, x_i)$ and moves to $z_{i+1} = g(z_i, x_i)$, starting from a fixed initial state z_1 . The words of B may have different lengths, and B may contain the empty word, so block-to-variable and variable-to-variable coding are both covered.

Information losslessness says nothing about when the input can be recovered: the decoder may have to wait for the whole output before any letter is known. A stronger requirement bounds that waiting time.

Definition 1.19 (Information lossless encoder (ILE), [2]). *An encoder E is information lossless (IL) if for every initial state $z_1 \in Z$ and every input $x_1^n \in A^n$ the triple*

$$(z_1, f(z_1, x_1^n), g(z_1, x_1^n))$$

determines x_1^n uniquely.

Definition 1.20 (Information lossless encoder of finite order (ILF), [2]). *An encoder E is information lossless of finite order if there exists a finite positive integer m such that for every $z_1 \in Z$ and every $x_1^m \in A^m$ the pair $(z_1, f(z_1, x_1^m))$ determines the first input letter x_1 uniquely.*

In other words, an ILF encoder can be decoded letter by letter with bounded delay: the letter x_1 is fixed once the output produced for the first m input letters has arrived, the past being summarized by the known state z_1 . This is what a decoder working on the fly needs, and incremental parsing (Definition 1.16) provides it: a phrase is emitted as the index of an earlier phrase and one letter, so the decoder recovers it at once by a walk in the trie of Definition 1.17, and the delay never exceeds the length of a phrase.

1.2.6 Compression ratio

Three of our four algorithms emit codewords of fixed length over $\mathcal{A}$, while the encoder of LZ78 ([2]) writes a bit stream whose codewords grow with the phrase index. The ratio may be therefore stated in two forms.

Definition 1.21 (Compression ratio, [1], [2]). *Following [1], for a scheme emitting N codewords of fixed length L_c over the same alphabet $\mathcal{A}$ for a source of $|S|$ letters, the compression ratio is*

$$\rho = \frac{N \cdot L_c}{|S|}.$$

Following [2], for a scheme emitting an encoded bit stream of total length L bits, the ratio is measured against the $|S| \log_2 \alpha$ bits needed to write the source,

$$\rho = \frac{L}{|S| \log_2 \alpha}.$$

Definition 1.22 (Compressibility, [2]). *Let $\rho_{E(s)}(x)$ denote the smallest ratio attainable on sequence x by an information-lossless encoder with at most s states. The compressibility of x is*

$$\rho(x) = \lim_{s \rightarrow \infty} \rho_{E(s)}(x).$$

It is a property of the single sequence x , not attained by any finite-state encoder and approached as the number of states grows.

Chapter 2

LZ77

2.1 Idea

The approach of [1] is to design a compression scheme that works without full prior knowledge of the source and still approaches the bounds attainable by fixed code book schemes that possess such knowledge.

It appears that the dictionary need not be transmitted if we use the following approach. Each word is encoded by the position of an earlier occurrence together with its length, so decoder utilises the part of the text it has already reconstructed as dictionary for rebuilding next coded word. The dictionary is therefore dynamic and derived from the codewords themselves.

The algorithm consists of a rule for parsing strings of symbols from a finite alphabet A into substrings, or words, whose lengths do not exceed a prescribed integer L_s (Definition 1.13), also do it having memory of last seen fixed amount of symbols (Definition 1.12). A coding scheme which maps these substrings sequentially into codewords of fixed length L_c (Definition 1.14), over the same alphabet A . The codeword consists of a triple $(pos, len, symbol)$, where pos points to an earlier occurrence inside the window which was already parsed, len is the length of the match found there, and $symbol$ is the letter that follows it in the source and broke the match.

The code is uniquely decipherable, and each codeword can be decoded as soon as it arrives: all codewords have the same length L_c , so the stream is cut without delimiters, and the decoder never has to look ahead.

2.1.1 Theoretical vs practical approach

If we want to compress some source, in the ideal scenario we should know its characteristics, such as 1) the alphabet, and 2) which sequences of symbols may occur and which may not. Of course, a source can be characterized in other ways as well, but here we want to show the difference between the case where we have information about the source and the case where we do not.

As a rule, the alphabet is available to us in any case and does not place a large burden on transmitting it to the other side (the decoder). Further information, however, is not always at hand: either there is no way to determine it in advance, or it carries too large a structure to be practical, or we are dealing with a universal approach, which by its nature does not presuppose knowledge of all the possible data it will have to work on.

In the LZ77 paper the authors want to present an approach to constructing a universal scheme (encoder and decoder) that neither relies on nor has any prior knowledge of the sources it works on, and yet achieves the same compression efficiency as schemes that have full information about those.

This is achieved as follows: for a given source S over an alphabet $\mathcal{A}$, we build an encoder with the parameters mentioned earlier ($L_c, \dots$), and an instruction is given on how to compute them when the algorithm is given as input only the size of the alphabet and the interval $[\rho_1, \rho_2]$ within which we want to achieve the compression of the source. The main problem of this approach, and the reason why it is theoretical, is that the resulting parameters are not technically realizable because of their enormous size.

Let's first describe with what representation of source knowledge we are going to deal and then show the flow of [1] paper.

A source can be defined by the finite set of strings over $\mathcal{A}$, that can not occur in it. Assuming we have this information, we can get also the number of distinct strings of length m it can emit, which is denoted as $\sigma(m)$, and the associated h -parameter is $h(m) = \frac{1}{m} \log_\alpha \sigma(m)$. The quantity $h(m)$ plays for such a source the role that entropy plays for a probabilistic one, but it is defined without probabilities (also mentioned in [1]). It is used here only to state the bounds of [1] and does not appear elsewhere in this work.

Section 3 of [1] proceeds in two steps. First it derives the compression attainable when σ and the h -parameters are known to the designer. Then it shows that this knowledge is not in fact needed: an encoder built for a prescribed range of compression ratios performs almost as well as one matched to the source itself, what gives us the desirable scheme.

The buffer length (Definition 1.12) is chosen to be an integer of the form below. We give the formula, which was also introduced in [1], not in order to use it, but to show how large in theoretical approach the numbers it produces are: the two sums count the words the source is able to emit in the worst case, and the buffer has to be large enough to hold all of them.

$$n = \sum_{m=1}^{\lambda} m\alpha^m + \sum_{m=\lambda+1}^l m\sigma(l) + (l+1)(N_{l+1} + 1), \quad (2.1)$$

$$N_{l+1} = \sum_{m=1}^{\lambda} (l-m)\alpha^m + \sum_{m=\lambda+1}^l (l-m)\sigma(l), \quad (2.2)$$

where $l = L_s - 1$ and $\lambda = \lfloor l \cdot h(l) \rfloor$ is the integer part of $\log_\alpha \sigma(l)$.

Theorem 2.1 (Theorem 1 of [1]). *If the buffer length n is chosen according to (2.1) for a source whose h -parameters are known, then the compression ratio ρ of Definition 1.21 satisfies*

$$\rho \leq h(L_s - 1) + \varepsilon(L_s), \quad \varepsilon(L_s) = \frac{1}{L_s - 1} \left(3 + 3 \log_\alpha (L_s - 1) + \log_\alpha \frac{L_s}{2} \right).$$

The term $\varepsilon(L_s)$ decreases as L_s grows, so the ratio approaches $h(L_s - 1)$, the h -parameter of the second largest word length the encoder handles.

Theorem 2.2 (Theorem 2 of [1]). *Let E_0 be an encoder designed for a prescribed range $[\rho_0, \rho_1]$ of compression ratios. For every source σ whose own matched encoder achieves a ratio $\rho(\sigma)$*

inside that range,

$$\rho_0(\sigma) \leq \rho(\sigma) + \Delta,$$

where Δ is fixed by the range alone and does not depend on σ .

Theorem 1 of [1] is proved under the assumption that the h -parameters are known. Theorem 2 removes it. Instead of matching a particular h , one designs a single encoder E_0 for a prescribed range (ρ_0, ρ_1) of compression ratios. For every source whose own matched encoder achieves a ratio inside that range, E_0 exceeds it by at most a fixed amount Δ , which does not depend on the source and shrinks as the buffer grows.

Even in such encoder, because of the term α^m , the buffer length (2.1) grows exponentially in L_s . The authors note that buffer length $n \simeq L_s \alpha^{hL_s}$, where h is the h -parameter of the source, $h(m) = \frac{1}{m} \log_\alpha \sigma(m) \in [0, 1]$. Even for the most favorable case of binary alphabet the requirement becomes impossible very quickly: $\alpha = 2$, $h = 0.8$, and $L_s = 64$ for the formula already gives n of the order of 10^{17} , not even closely fitting any addressable memory.

Our implementation does not follow the aim to fit in certain bounds on compression, but rather practical observation of how good it works for feasible sizes on working buffer and corresponding to it L_s . At this point it is natural, that we want to make buffer length as big as hardware allows it, but in regard to L_s for different sources, where resulting factors are small — it can be better to maintain small size of L_s in order to speed up the algorithm. We see that our practical implementation differs in goals of authors, who were trying to present universal coding scheme.

2.1.2 Pseudocode

The only nontrivial step is finding the factor. The encoder has the window $B[1 \dots n - L_s]$ on one side and the unparsed part starting at $n - L_s + 1$ on the other, and it tries every position of the parsed part of the window in turn: for each i it compares symbol by symbol until a mismatch, and keeps the i giving the longest agreement with $B[n - L_s + 1 \dots n]$. This is the reproducible extension of Definition 1.15, and it is computed by REPRODUCIBLEEXTENSION in Algorithm 1. The search is naive in the sense of computation time, and Chapters 4 and 5 show how to avoid it.

In our implementation we use ring for window representation, so $B[k]$ is the k -th cell after the head of the ring.

Algorithm 1 LZ77 encoder with a sliding window

```
1: require the source  $S$  over an alphabet of size  $\alpha$ , the buffer length  $n$ , the maximal word  
   length  $L_s$   
2: output the codeword stream  $C_1 C_2 \dots$ , each codeword of  $L_c$  symbols of  $\mathcal{A}$   
  
3: function MAKEPARAMETERS( $\alpha, n, L_s$ )  
4:    $L_p \leftarrow \lceil \log_\alpha(n - L_s) \rceil$ ,  
5:    $L_l \leftarrow \lceil \log_\alpha L_s \rceil$   
6:   return ( $\alpha, n, L_s, L_p, L_l, L_c = 1 + L_p + L_l$ )  
  
7: function REPRODUCIBLEEXTENSION( $B, j, m$ )  $\triangleright m = n - 1, j = n - L_s$   
8:    $(p^*, l^*) \leftarrow (1, 0)$   
9:   for  $i \leftarrow 1$  to  $j$  do  
10:     $l \leftarrow 0$   
11:    while  $l < m - j$  and  $B[i - 1 + l] = B[j + l]$  do  
12:       $l \leftarrow l + 1$   
13:    if  $l \geq l^*$  then  $\triangleright$  on equality we take rightmost candidate  
14:       $(p^*, l^*) \leftarrow (i, l)$   
15:   return  $(p^*, l^*)$   
  
16: procedure ENCODE( $S, \alpha, n, L_s$ )  
17:   MAKEPARAMETERS( $\alpha, n, L_s$ )  
18:    $B \leftarrow$  ring buffer of  $n$  zeros  
19:   for  $i \leftarrow 0$  to  $L_s - 1$  do  
20:      $B[n - L_s + i] \leftarrow \text{NEXTSYMBOL}(S)$   $\triangleright$  zeros after the end of  $S$   
21:   while not all of  $S$  has been encoded do  
22:      $(p, l) \leftarrow \text{REPRODUCIBLEEXTENSION}(B, n - L_s, n - 1)$   
23:      $C_{i1} \leftarrow \text{RADIX}(p - 1, \alpha, L_p)$   
24:      $C_{i2} \leftarrow \text{RADIX}(l, \alpha, L_l)$   
25:      $C_{i3} \leftarrow B[n - L_s + l]$   
26:     emit  $C_{i1} \cdot C_{i2} \cdot C_{i3}$   
27:     for  $k \leftarrow 1$  to  $l + 1$  do  
28:        $s \leftarrow \text{NEXTSYMBOL}(S)$   
29:       SHIFTIN( $B, s$ )
```

Algorithm 2 LZ77 decoder

```
1: require the codeword stream  $C_1C_2\dots$ , the window length  $n - L_s$ , the field widths  
    $L_p, L_l, L_c$ , the alphabet size  $\alpha$ , source length  $|S|$   
2: output the source string  $S$   
  
3: procedure DECODENEXT( $C$ )  
4:   require  $|C| = L_c$   
5:    $p \leftarrow \text{FROMRADIX}(C[1, \dots, L_p], \alpha) + 1$   
6:    $l \leftarrow \text{FROMRADIX}(C[L_p + 1, \dots, L_p + L_l], \alpha) + 1$   $\triangleright$  the field holds  $l - 1$   
7:   require  $1 \leq p \leq n - L_s$  and  $1 \leq l \leq L_s$   
8:   for  $k \leftarrow 1$  to  $l - 1$  do  
9:      $\text{SHIFTIN}(D, D[p - 1])$   $\triangleright$  read and write move together  
10:   $\text{SHIFTIN}(D, C[L_c])$   $\triangleright$  the mismatch letter  
11:  return  $D[n - L_s - l \dots n - L_s - 1]$   
  
12: procedure DECODE( $stream$ )  
13:   $D \leftarrow$  ring buffer of size  $n - L_s$  filled with zeros  
14:   $out \leftarrow$  empty list  
15:  for each block  $C$  of  $L_c$  symbols of  $stream$  do  
16:     $out.append(\text{DECODENEXT}(C))$   
17:  return the first  $|S|$  symbols of  $out$ 
```

The encoder starts with a window of zeros, so the very first word is found by the same rule as all the others and no special case is needed. Each iteration emits exactly one codeword and consumes $l + 1$ symbols: the l matched ones and the mismatch letter. Since l may be zero, the encoder always moves forward, and since the match is sought only against the window, l never exceeds $L_s - 1$, so the word fits into the length field.

The decoder is mirroring an encode, works on buffer of size $n - L_s$ in order to remember symbols which are needed to decode next codeword, reconstruct a word symbol by symbol in order to stay in the size of a buffer and shift new decoded symbols in order to be able to parse even those substrings which were not present in previous buffer state.

2.1.3 Additional notes

The problem with algorithm proposed here is that, first, it will represent the naive way (REPRODUCIBLEEXTENSION) of finding best factors in current buffer, but this can be fixed, with other factorization approaches, for example the ones in Chapters 4 and 5, though they won't be seen right now, but rather after next chapter where we are going to describe the second main algorithm of this work.

2.2 Correctness

The correctness of the scheme rests on the following facts:

1. the parsing always makes progress and never produces a word too long for the codeword to hold,

2. the decoder can be kept in the same state as the encoder.

2.2.1 The parsing is well defined

For every i , the word S_i produced by REPRODUCIBLEEXTENSION in Algorithm 1 satisfies $1 \leq l_i \leq L_s$. This follows from the fact that we always add one mismatch symbol and the reproducible extension is searched only for last $L_s - 1$ symbols of buffer.

The fields of the codeword are wide enough. Since $1 \leq l_i \leq L_s$, the value $l_i - 1$ admits a radix- α representation in $\lceil \log_\alpha L_s \rceil$ symbols; since the pointer refers to a position of the window, $1 \leq p_i \leq n - L_s$, and $p_i - 1$ fits into $\lceil \log_\alpha (n - L_s) \rceil$ symbols. Together with the single symbol C_{i3} this gives L_c .

2.2.2 The decoder reconstructs the source

The decoder keeps a buffer D_i of $n - L_s$ symbols, initially loaded with $n - L_s$ zeros. Having read p_i and l_i from the fields of C_i , it applies $l_i - 1$ shifts while feeding the decoded symbols from reproducible extension, and then one more shift feeding in the last symbol C_{i3} .

We claim that before each codeword is decoded, the decoder buffer D_i holds exactly the same $n - L_s$ symbols as the encoder window did when that codeword was produced. This holds by induction on i . For $i = 1$ both buffers are filled with $n - L_s$ zeros by construction. Assume it holds for i . The first $l_i - 1$ symbols of S_i are read off positions $p_i, p_i + 1, \dots$ of the encoder window, and the decoder reads them off the same positions of D_i , which is equal to it by assumption. The last symbol is transmitted explicitly as C_{i3} . Both sides then shift in the same l_i symbols, so the buffers agree again, and the claim holds for $i + 1$.

2.3 Complexity

The buffer B is a ring rather than an array that is physically shifted. A shift would cost $\mathcal{O}(n)$ per word and would dominate the running time. The ring keeps a head index instead and a shift costs $\mathcal{O}(1)$, at the price of every access going through a modulo. Since the head advances by one at a time, the modulo is replaced by a single comparison.

The cost of encoding is the cost of REPRODUCIBLEEXTENSION in Algorithm 1, since everything else in the main loop is done once per word. That function tries every position of the parsed part of window, so its outer loop runs $n - L_s$ times, and for each candidate the inner loop compares symbols until a mismatch. No candidate can match further than the best one, so each of them costs at most l_i comparisons and one word costs at most $(n - L_s) l_i$.

The words partition the source, so $\sum_i l_i = |S|$ and the total is

$$\sum_i (n - L_s) l_i = (n - L_s) \sum_i l_i = \mathcal{O}((n - L_s) \cdot |S|).$$

Since $n \geq 2 \cdot L_s$ it would be correct to write as $\mathcal{O}(n \cdot |S|)$.

Decoding is linear. A word of length l_i is rebuilt by l_i shifts in Algorithm 2, and the lengths again sum to $|S|$, giving $\Theta(|S|)$ in total.

The memory usage is $\Theta(n)$ for the encoder and $\Theta(n - L_s)$ for the decoder, independent of the length of the source. Both buffers are implemented as ring buffers, so one shift costs $\mathcal{O}(1)$.

Chapter 3

LZ78

3.1 Idea

In [2] work the authors look at the compression problem from the side of finite-state machines (Definition 1.18), and more precisely of the ILF ones (Definition 1.20). There is no source model here at all: the object of study is one individual sequence x , and what replaces the source is the class of machines against which the comparison is made, namely the lossless finite-state encoders with at most s states.

The idea is to prove the existence of an encoder, bounded in the number of states, whose compression ratio is bounded from below and from above tightly enough by quantities related to $c(x_1^n)$, the largest number of distinct phrases into which the string x_1^n can be parsed. Both bounds rest on properties of the encoder, but on different ones. The lower bound is proved under the assumption of information losslessness (*IL*, Definition 1.19), which is wider than *ILF*. The upper bound is proved by constructing a concrete encoder, and that encoder belongs to the narrower class *ILF* (Definition 1.20).

Theorem 3.1 (Theorem 1 and Corollary 1 of [2]). *For every sequence x , the compressibility of Definition 1.22 satisfies*

$$\rho(x) \geq \limsup_{n \rightarrow \infty} \frac{c(x_1^n) \log c(x_1^n)}{n \log \alpha}.$$

Theorem 3.2 (Theorem 2 of [2]). *For every block length n there exists a finite-state encoder E with $s(n)$ states, implementing a block-to-variable code, whose ratio on a block satisfies*

$$\rho_E(x_1^n) \leq \frac{c(x_1^n) + 1}{n \log \alpha} \log (2\alpha(c(x_1^n) + 1)),$$

and on an infinite sequence exceeds $\rho(x)$ by an amount that tends to zero as n grows.

Corollary 3.3 (Theorems 1 and 2 and Corollary 1 of [2]). *Let E be the encoder of Theorem 3.2 and let c stand for $c(x_1^n)$. Since E is information lossless and has finitely many states, the bound of Theorem 3.1 applies to it as well, so its ratio is squeezed from both sides:*

$$\limsup_{n \rightarrow \infty} \frac{c \log c}{n \log \alpha} \leq \limsup_{n \rightarrow \infty} \rho_E(x_1^n) \leq \limsup_{n \rightarrow \infty} \frac{(c+1) \log (2\alpha(c+1))}{n \log \alpha}.$$

The two sides are close to each other. The upper bound $\frac{(c+1) \log (2\alpha(c+1))}{n \log \alpha} = \frac{(c+1) \log (c+1)}{n \log \alpha} + \frac{(c+1) \log (2\alpha)}{n \log \alpha}$, so the bounds differ by $\frac{(c+1) \log (2\alpha)}{n \log \alpha}$, where 2α is a constant and what is left is

$\frac{c+1}{n}$, by (6) of [2] we know that $c(x_1^n) \leq \frac{n \log \alpha}{(1-\epsilon_n) \log n}$, so this is of order $\frac{1}{\log n}$ and vanishes as n grows.

We omit the proofs of these theorems and refer the reader to Sections 2 and 3 of [2].

Such a dictionary grows infinitely if given infinite input, because incremental parsing gives infinite number of substrings in such case and moreover $c(x_1^\infty)$ is even greater, thus in order to stay with the finite-state, finite-memory machine it is proposed to divide input into blocks of fixed length, so in each such block dictionary first is reset and then again built from scratch. Here the codewords are not of fixed length. The j -th phrase of a block is sent as

$$I_j = \pi(j) \cdot \alpha + a_j \quad L_j = \lceil \log_2(j\alpha) \rceil \quad (3.1)$$

where $\pi(j)$ is from Definition 1.17 and gives the index of the phrase being extended, a_j is the appended symbol, L_j is the width of the field in bits, and j is the number of the phrase within the current block. The width of codeword grows with j , since the field has to hold the index of any phrase seen so far, and it starts over from $\lceil \log_2 \alpha \rceil$ at every block. The code is uniquely decipherable, and each codeword can be decoded as soon as it arrives.

3.1.1 Pseudocode

In the pseudocode below D is the trie of Definition 1.17. $\text{CHILD}(D, v, s)$ returns the index of the child of node v along the edge labeled s , or 0 if there is none. $\text{ADD}(D, v, s)$ creates that child as a new leaf, and $\text{PARENTSEQUENCE}(D, v)$ returns the phrase spelled by node v , obtained by walking up to the root and reversing the symbols collected on the way.

Algorithm 3 LZ78 encoder with block reset

```
1: require the source  $S$  over an alphabet of size  $\alpha$ , the block length  $n$ 
2: output the codeword stream, the  $j$ -th codeword of a block written in
   CODEWORDWIDTH( $\alpha, j$ ) bits

3: procedure EMITPHRASE( $\pi, a, \alpha, j$ )
4:   write  $\pi \cdot \alpha + a$  in CODEWORDWIDTH( $\alpha, j$ ) bits

5: procedure ENCODEBLOCK( $S, \alpha, n, position$ )
6:    $end \leftarrow \min(position + n, |S|)$  ▷ the last block may be shorter
7:   RESET( $D$ )
8:    $j \leftarrow 1$ 
9:    $current\_node \leftarrow 0$ 
10:  while  $position < end$  do
11:     $s \leftarrow S[position]$ 
12:     $position \leftarrow position + 1$ 
13:     $next\_node \leftarrow \text{CHILD}(D, current\_node, s)$  ▷ 0 when the edge is absent
14:    if  $next\_node \neq 0$  then
15:       $current\_node \leftarrow next\_node$  ▷ the walk continues
16:    else
17:      EMITPHRASE( $current\_node, s, \alpha, j$ )
18:      ADD( $D, current\_node, s$ )
19:       $current\_node \leftarrow 0$ 
20:       $j \leftarrow j + 1$ 
21:    if  $current\_node \neq 0$  then
22:      ▷ rollback on one step in order to add a mismatch symbol
23:      EMITPHRASE( $D.parent[current\_node], D.last[current\_node], \alpha, j$ )
24:      ▷  $D.last[node\_number]$  returns symbol of this node

23: procedure ENCODE( $S, \alpha, n$ )
24:    $position \leftarrow 0$ 
25:   while  $position < |S|$  do
26:     ENCODEBLOCK( $S, \alpha, n, position$ )
```

Algorithm 4 LZ78 decoder

```
1: require the codeword stream, the source length  $|S|$ , alphabet of size  $\alpha$ , the block length  
    $n$   
2: output the source string  $S$   
  
3: procedure DECODE( $stream, |S|, \alpha, n$ )  
4:    $out \leftarrow$  empty list  
5:   while  $|out| < |S|$  do  
6:      $block\_length \leftarrow \min\{n, |S| - |out|\}$   $\triangleright$  the last block may be shorter  
7:     RESET( $D$ )  
8:      $j \leftarrow 1$   
9:      $decoded\_length \leftarrow 0$   
10:    while  $decoded\_length < block\_length$  do  
11:       $I \leftarrow$  next  $\lceil \log_2(j\alpha) \rceil$  bits of  $stream$   
12:       $(\pi, a) \leftarrow (\lfloor I/\alpha \rfloor, I \bmod \alpha)$   
13:       $y \leftarrow$  PARENTSEQUENCE( $D, \pi$ ) $\cdot a$   $\triangleright$  walk up the trie from node  $\pi$ , then reverse  
14:       $out.append(y)$   
15:       $decoded\_length \leftarrow decoded\_length + |y|$   
16:      ADD( $D, \pi, a$ )  
17:       $j \leftarrow j + 1$   
18:    return  $out$ 
```

The encoder starts each block with an empty dictionary, so the first phrase of a block is always a single symbol and no special case is needed. Inside the block the walk down the trie costs one step per input symbol: either the edge exists and the walk continues, or it does not and the phrase ends there, which is why the parsing never looks back and never compares anything twice. Every emitted phrase adds exactly one node to the dictionary, so the phrase counter j and the number of nodes stay equal, and both sides can derive the field width L_j from it without transmitting anything.

The decoder is mirroring the encoder. It keeps the same dynamic dictionary and the same counter, so it computes L_j itself, reads that many bits, recovers I_j , splits it into $\pi(j)$ and a_j by division and remainder by α , and rebuilds the phrase by walking the same path as encoder, and of course clears its dictionary after n rebuilt symbols.

3.2 Correctness

The correctness of the scheme rests on the following facts:

1. the parsing always makes progress and never produces a word too long for the codeword to hold,
2. the decoder can be kept in the same state as the encoder.

We write $y_1 y_2 \dots y_p$ for the phrases of one block, $l_j = |y_j|$, D_j is dictionary after j phrases have been processed.

3.2.1 The parsing is well defined

By construction in Algorithm 3, $y_j = y_{\pi(j)}a_j$, where $y_{\pi(j)} \in D_{j-1}$ and $y_{\pi(j)}a_j \notin D_{j-1}$. Hence

$$y_j \notin D_{j-1}, \quad \pi(j) < j, \quad l_j = l_{\pi(j)} + 1 \geq 1$$

so the phrases of a block are distinct pairwise, every phrase without its last symbol is an earlier one, each encoding step consumes at least one symbol. This is exactly the incremental parsing structure. Since $D_0 = \emptyset$ we have $\pi(1) = 0$ and $l_1 = 1$, and since $\sum_j l_j = n$ a block is parsed in at most n steps.

The fields of the codeword are wide enough:

$$0 \leq I_j = \pi(j)\alpha + a_j \leq (j-1)\alpha + (\alpha-1) = j\alpha - 1 < 2^{L_j} \quad L_j = \lceil \log_2 j\alpha \rceil$$

Here we recompute codeword length every step, starting from $L_1 = \lceil \log_2 \alpha \rceil$.

3.2.2 The decoder reconstructs the source

The codewords have different lengths and no delimiters, so decipherability needs the boundaries to be computable. We maintain counter which shows us the positions of bits of the next codeword:

$$k_0 = 0, \quad k_j = k_{j-1} + \lceil \log_2(j\alpha) \rceil,$$

so the j -th codeword occupies bits $k_{j-1} + 1, \dots, k_j$.

Let D'_j denote the decoder dictionary. We claim $D'_j = D_j$ for all j , by induction. Both are empty at $j = 0$. Given $D'_{j-1} = D_{j-1}$, the decoder reads I_j from the bits above and recovers

$$\pi(j) = \lfloor I_j / \alpha \rfloor, \quad a_j = I_j \bmod \alpha,$$

which is exact since $0 \leq a_j < \alpha$. As $\pi(j) < j$, the entry $y_{\pi(j)}$ lies in $D_{j-1} = D'_{j-1}$, so $y_j = y_{\pi(j)}a_j$ is rebuilt and inserted on both sides, giving $D'_j = D_j$.

Blocks agree because both sides count source symbols. The tail rule(that we cut the on n symbols) makes $\sum_j l_j = n$ hold exactly.

3.3 Complexity

Our implementation descends the trie while reading the input, so every source symbol costs one dictionary operation: it either continues the descent or ends a phrase. The phrases partition the block, $\sum_j l_j = n$, so a block costs $\mathcal{O}(n)$ operations and the whole source costs $\mathcal{O}(|S|)$. We store children of a node in a hash map where one operation costs $\mathcal{O}(1)$ on average, giving $\mathcal{O}(|S|)$ expected time.

Decoding is linear: a word of length l_j is reconstructed by walking up the tree in $\mathcal{O}(l_j)$ steps, and $\sum_j l_j = n$, giving $\mathcal{O}(|S|)$ in total. If we take into consideration working time optimization, we can reconstruct trie, so it stores in each node the string this node represents, what would give us $\mathcal{O}(1)$ time for reconstruction of a single word, but this would require more working space up to $\mathcal{O}(n^2)$ in dictionary and time complexity $\mathcal{O}(N)$, where N is number of codewords.

The memory usage is the size of one dictionary. The phrases of a block are distinct and their lengths sum to n , each phrase gives one more node in D , so both encoder and decoder use $\mathcal{O}(n)$ entries in worst case.

Chapter 4

LZ77: Chen–Puglisi–Smyth variant

4.1 Idea

4.1.1 From online to offline

Both schemes seen so far factor the source against a bounded history: *LZ77* against a window of $n - L_s$ symbols, *LZ78* against the dictionary of the current block and it was natural since those implementations fit for online coding. The algorithms of [13] drop that restriction and factor the string as a whole, so that every factor is matched against the entire prefix preceding it. This is the *LZ* factorization in the sense of Definition 1.4: $x = w_1 w_2 \cdots w_k$, where each w_j is either a letter not occurring in $w_1 \cdots w_{j-1}$, or the longest substring occurring at least twice in $w_1 \cdots w_j$. Of the several algorithms proposed in [13] we implement the basic one, *CPS1a*.

The motivation is different from that of [1]. Compression is no longer the only purpose: the factorization of a whole string is what they are focusing on and for this application full string is available. This is why the authors are willing to spend linear extra space on index structures, which a streaming compressor could not afford.

4.1.2 *CPS1a* approach description

Before proceeding with following description it is highly recommended to familiarize oneself with the definitions and examples in Section 1.2.

We will start description first showing what we want to compute, then how we do it. The factorization of string is the part that changes in *LZ77* offline encoding. *CPS1a* takes two arrays as input and computes the answer in one left-to-right scan. In order to achieve this we need SA and LCP of Definition 1.7 and Definition 1.8.

The *CPS1a* algorithm computes two arrays POS and LEN from Definition 1.6.

Note that after we compute such POS and LEN it is required only one left-to-right run in order to form our triples of codeword in order to form the same form of coded output as did in *LZ77* chapter 2.

Now, how can we compute these two arrays POS and LEN from Definition 1.6. The POS[i] and LEN[i] tell us that substring $S[i \dots i + \text{LEN}[i] - 1]$ was already seen at $S[\text{POS}[i] \dots \text{POS}[i] + \text{LEN}[i] - 1]$. We can view it as fact that suffixes of S that start at i and POS[i] *share the same*

prefix of length $\text{LEN}[i]$ what is very similar to LCP Definition 1.8. This gives the property the whole algorithm rests on. Suppose several positions of S share the same factor w of length l , that is, $S(i, i + l - 1) = w$ for each of them. Then the suffixes starting at those positions all begin with w , and in SA they must occupy a contiguous range. Indeed, if some suffix u is placed between two suffixes that begin with w , then u is lexicographically between them, and since comparison proceeds symbol by symbol from the left, u must agree with them on the first l symbols as well, so u begins with w too. Nothing that does not begin with w can therefore be placed inside the range.

This leads us to the following lemma:

Lemma 4.1 (Rises and falls of LCP, [13]). *Let $1 \leq j \leq n$.*

(O1) *If $\text{LCP}[j] < \text{LCP}[j + 1]$, then the suffixes $\text{SA}[j]$ and $\text{SA}[j + 1]$ share a prefix of length $\text{LCP}[j + 1]$ that no earlier entry of SA shares. A rise thus opens a block of suffixes with a common beginning on j .*

(O2) *If $\text{LCP}[j] > \text{LCP}[j + 1]$, then the suffixes $\text{SA}[j - 1]$ and $\text{SA}[j]$ share a prefix of length $\text{LCP}[j]$ that no later entry shares. A fall thus closes such a block at j .*

Proof of Lemma 4.1. For (O1), let $\text{LCP}[j] < \text{LCP}[j + 1] = l$. The suffixes $\text{SA}[j]$ and $\text{SA}[j + 1]$ share a prefix w of length l by the definition of LCP. Suppose some earlier entry $\text{SA}[k]$, $k < j$, also began with w . Then all entries between k and $j + 1$ would begin with w as well, since the suffixes starting with w form a contiguous range, and in particular $\text{SA}[j - 1]$ and $\text{SA}[j]$ would share w , giving $\text{LCP}[j] \geq l$, which contradicts the assumption.

Part (O2) is symmetric. Let $\text{LCP}[j] = l > \text{LCP}[j + 1]$, so $\text{SA}[j - 1]$ and $\text{SA}[j]$ share a prefix w of length l . If some later entry $\text{SA}[k]$, $k > j$, began with w , then by the same contiguity $\text{SA}[j]$ and $\text{SA}[j + 1]$ would share w , giving $\text{LCP}[j + 1] \geq l$, again a contradiction. $\square$

So every repeated factor of S appears in SA as a block of neighbouring entries, and the length of the repetition is recorded in LCP at exactly the positions of that block. Finding all repeated factors is then a matter of finding these blocks, which is what Items (O1) and (O2) make possible.

Reading the same string through SA shows where the blocks open and close:

j	$\text{SA}[j]$	suffix	$\text{LCP}[j]$	
1	8	<i>a</i>	−1	rise, a block opens
2	3	<i>aababa</i>	1	
3	6	<i>aba</i>	1	rise, a deeper block opens
4	1	<i>abaababa</i>	3	
5	4	<i>ababa</i>	3	fall, both blocks close
6	7	<i>ba</i>	0	rise, a block opens
7	2	<i>baababa</i>	2	
8	5	<i>baba</i>	2	fall, the block closes
9	—	—	−1	

Table 4.1: Rises and falls of LCP for $x = abaababa$.

Two things follow from the table. A fall can close several blocks at once, as at $j = 5$, where the ranges of level 3 and of level 1 end together. And blocks are nested, never partly

overlapping, since a longer common prefix means a shorter range. So while scanning SA left to right we may have several blocks open at the same time, and they close in the reverse order of opening — which is exactly what a stack is for.

4.1.3 Pseudocode

Now we are going to show pseudocode, after that we proceed with algorithm elaboration.

Algorithm 5 *CPS1a*: computing POS and LEN and factorization computation

```

1: function ASSIGN( $p, q, l$ ) ▷ returns  $\min(p, q)$ 
2:   if  $p < q$  then
3:     POS[ $q$ ]  $\leftarrow p$ ,   LEN[ $q$ ]  $\leftarrow l$ 
4:     return  $p$ 
5:   else
6:     POS[ $p$ ]  $\leftarrow q$ ,   LEN[ $p$ ]  $\leftarrow l$ 
7:     return  $q$ 

8: procedure COMPUTEPOSLLEN(SA, LCP,  $n$ )
9:   POS[0]  $\leftarrow -1$ ,   LEN[0]  $\leftarrow -1$ 
10:  for  $i \in 1 \dots n$  do
11:    POS[ $i$ ]  $\leftarrow i$ ,   LEN[ $i$ ]  $\leftarrow 0$ 
12:  if  $n < 2$  then
13:    return
14:  LCP[1]  $\leftarrow -1$ 
15:  LCP[ $n + 1$ ]  $\leftarrow -1$ 
16:   $T \leftarrow$  empty stack
17:   $i_1 \leftarrow 1$ ,    $i_2 \leftarrow 2$ ,    $i_3 \leftarrow 3$ 
18:  while  $i_3 \leq n + 1$  do
19:    while LCP[ $i_2$ ]  $\leq$  LCP[ $i_3$ ] do ▷ until fall in common prefix length detected we
    proceed proceed
20:    PUSH( $T, i_1$ )
21:     $i_1 \leftarrow i_2$ ,    $i_2 \leftarrow i_3$ ,    $i_3 \leftarrow i_3 + 1$ 
22:     $l \leftarrow$  LCP[ $i_2$ ]
23:     $q \leftarrow$  ASSIGN(SA[ $i_1$ ], SA[ $i_2$ ],  $l$ )
24:    while LCP[ $i_1$ ] =  $l$  do
25:       $i_1 \leftarrow$  POP( $T$ )
26:       $q \leftarrow$  ASSIGN(SA[ $i_1$ ],  $q, l$ )
27:    SA[ $i_1$ ]  $\leftarrow q$  ▷ the closed group is now a single entry
28:    if  $i_1 > 1$  then
29:       $i_2 \leftarrow i_1$ 
30:       $i_1 \leftarrow$  POP( $T$ ) ▷ step one level outwards
31:    else
32:       $i_2 \leftarrow i_3$ 
33:       $i_3 \leftarrow i_3 + 1$  ▷ sentinel reached, nothing is open

```

Algorithm 6 *CPS1a*: continuation

```
1: function FACTORIZE( $S, n, \text{POS}, \text{LEN}$ )
2:    $\text{triples} \leftarrow$  empty list,    $i \leftarrow 1$ 
3:   while  $i \leq n$  do
4:      $l \leftarrow \text{LEN}[i]$ 
5:     if  $i + l > n$  then
6:        $l \leftarrow n - i$  ▷ leave room for the literal
7:     if  $l > 0$  then
8:        $\text{triples.append}(\text{POS}[i], l, S[i + l])$ 
9:     else
10:       $\text{triples.append}(1, 0, S[i])$  ▷ no earlier occurrence, the pointer is unused
11:     $i \leftarrow i + l + 1$ 
12:  return  $\text{triples}$ 
```

The three pointers satisfy $i_1 < i_2 < i_3$ throughout. The inner loop at lines 15-18 advances them while LCP does not fall, pushing i_1 each time, so the stack accumulates the left boundaries of the blocks currently open, innermost on top. The fall at line 19 gives the level l of the block that ends at i_2 . The loop at lines 28-30 pops every boundary whose level equals l , calling ASSIGN for each; the running value q keeps the smallest text position met so far, so every call points a larger position at a smaller one. Line 31 writes q back into $\text{SA}[i_1]$, which lets the closed block be treated as one entry, and lines 32-34 step one level outwards.

The pointer stored for a position is some earlier occurrence, not necessarily the leftmost one. The authors note that this is harmless: neither the computation inside encoder nor decompression needs the leftmost occurrence, only an occurrence to the left. Our decoder confirms this, since it copies from wherever the pointer says both in this implementation and *LZ77* from previous chapters.

4.1.4 Encoder and decoder of *CPS1a* version

Encoder proceeds as follows.

1. Build SA and LCP for the whole source.
2. Scan them once as described above, filling arrays POS and LEN so that for every position i the substring of length $\text{LEN}[i]$ starting at i also occurs at $\text{POS}[i] < i$. The flow of *CPS1a* is going to be described later.
3. Walk the source from left to right. At position i take $l_i = \text{LEN}[i]$, emit the factor $(\text{POS}[i], l_i, x[i + l_i])$, and move to $i + l_i + 1$. The last component is the mismatch letter, so l_i is shortened when the factor would otherwise reach the end of the source.
4. Encode each factor as a codeword $C_i = C_{i1}C_{i2}C_{i3}$ of fixed length $L_c = 1 + L_p + L_l$, where $L_p = \lceil \log_\alpha |S| \rceil$ and $L_l = \lceil \log_\alpha (\max_i l_i + 1) \rceil$.

The pointer field is $\lceil \log_\alpha |S| \rceil$ wide, since a factor may now refer anywhere in the source rather than into a window of fixed length. And the length field is sized from the longest factor actually produced, which is known once the factorization is complete, rather than from

a parameter fixed in advance as L_s was in *LZ77*. This costs one pass over the factors and saves on each codeword whenever the source has no long repetitions.

The decoder is the same as in *LZ77* (see Chapter 2). It appends l_i symbols copied from position $\text{POS}[i]$ of the output built so far, then appends the literal. Copying proceeds symbol by symbol, so a factor overlapping its own previous occurrence is handled correctly, exactly as in Section 2.2. However, we no longer need a ring buffer: the whole decoded text has to be kept, because a pointer may reach arbitrarily far back.

4.2 Correctness

Throughout this section S is the source, $n = |S|$, and SA, LCP are its suffix and LCP arrays, indexed from 1 and extended by the sentinel values $\text{LCP}[1] = \text{LCP}[n+1] = -1$. We first prove Lemma 4.1, then that the scan fills POS and LEN correctly, and finally that the decoder recovers S from the factors built from them.

Recall from Algorithm 5 that entries are written by

$$\text{ASSIGN}(p, q, l) = \begin{cases} \text{POS}[q] \leftarrow p, \text{LEN}[q] \leftarrow l, & \text{for } p < q, \\ \text{POS}[p] \leftarrow q, \text{LEN}[p] \leftarrow l, & \text{for } p \geq q, \end{cases}$$

which returns $\min(p, q)$. Two consequences are used throughout: the entry written always belongs to the larger of the two text positions and points to the smaller, and the returned value is the smallest text position met so far in the group being closed.

Lemma 4.2 (Stack invariant). *At every entry to the forward loop the following holds. The pointers satisfy $i_1 < i_2 < i_3$, and the stack contains, from bottom to top, positions $j_1 < j_2 < \dots < j_m < i_1$ with $\text{LCP}[j_1] \leq \text{LCP}[j_2] \leq \dots \leq \text{LCP}[j_m] \leq \text{LCP}[i_1]$.*

Proof. Initially $i_1 = 1, i_2 = 2, i_3 = 3$ and the stack is empty. As we set $\text{LCP}[1] = -1$ which is smaller than every other entry, position 1 can never be popped by the backtracking loop and acts as a sentinel at the bottom, so each backtracking procedure stops on encountering it.

The forward loop runs while $\text{LCP}[i_2] \leq \text{LCP}[i_3]$. Each turn pushes i_1 and shifts the three pointers one place to the right, so the new i_1 is the old i_2 . The pushed position is therefore smaller than the new i_1 , and the relation between their LCP values is the loop condition tested on the previous turn. Invariant holds.

The loop stops when $\text{LCP}[i_2] > \text{LCP}[i_3]$, that is, at a fall. By (O2), i_2 is the right boundary of a group of level $l = \text{LCP}[i_2]$. The backtracking loop pops exactly those positions whose LCP value equals l , which by the invariant are the left boundaries of the groups of that level, and stops at the first position with a strictly smaller value. All the popped groups are closed, so removing them preserves the invariant.

The last step of the iteration moves one level outwards. Suppose $i_1 > 1$, then $\text{LCP}[i_1] < l$, so i_1 is the left boundary of a wider group that contains the one just closed and is not finished yet. The group that was closed has been replaced by the single entry $\text{SA}[i_1]$, holding its smallest text position, so the algorithm can now treat it as one element: it sets $i_2 \leftarrow i_1$ and takes the next left boundary off the stack into i_1 . The outer loop then checks whether this wider group has to be closed too, note: i_1 and i_2 decreased and still $i_1 < i_2 < i_3$. If instead $i_1 = 1$, the sentinel has been reached: the stack is empty and no group is open, so the scan simply continues from i_3 . $\square$

Lemma 4.3 (Every assignment is valid). *Each time ASSIGN writes $\text{POS}[i] = p$ and $\text{LEN}[i] = l$, we have $p < i$ and $S(p, p + l - 1) = S(i, i + l - 1)$.*

Proof. That $p < i$ is immediate from the definition of ASSIGN, which writes into the larger of its two arguments. For the equality, both arguments are text positions taken from entries of SA lying in the group closed at i_2 , and $l = \text{LCP}[i_2]$ is the level of that group, so by Definition 1.9 the corresponding suffixes share a prefix of length l .

The arguments of ASSIGN are not always the original entries of SA, which was given at the start: after closing a group the algorithm overwrites $\text{SA}[i_1] \leftarrow q$, so a later call may read a value that was put there by an earlier step. This is safe, because the value q is the smallest text position of the group that was closed, and every suffix of that group, q included, starts with the same l symbols. The group has been closed before the ones containing it, so any later call using this entry works at level $l' \leq l$, and $l' < l$ from construction. $\square$

Every position is covered

It remains to see that the scan leaves nothing out. Take a position i at which some factor has an earlier occurrence, and let l be the length of the longest one. Suffix i and that earlier suffix start with the same l symbols, so both belong to a group of level l , and since l is the longest match, to no group of a higher level.

That group is certain to be processed. It is closed at the position where LCP drops below l , and such a drop always comes, with exception at the sentinel $\text{LCP}[n + 1] = -1$. When it is closed, the backtracking loop runs over all its members, so ASSIGN is called with i as one of its two arguments. The other argument is a smaller position, because i has an occurrence to its left, so i is the larger one and receives the entry. By Lemma 4.3 what it receives is a real match, and its length is l , the level of the inner-most group holding i .

If instead i has no earlier occurrence, then suffix i shares no beginning with anything before it and belongs to no group at all. It is never the larger argument of an assignment, so it keeps $\text{LEN}[i] = 0$ and $\text{POS}[i] = i$.

The last thing to cover is the termination of algorithm. At each iteration either i_3 increases or the stack loses at least one element, and at least one entry of POS is written, so we have at most $O(n)$ total iterations.

4.3 Complexity

Precalculation of SA and LCP takes $\Theta(n)$, e.g., [16] and [15]. The scan is linear: by Lemma 4.2 every position of SA is pushed at most once and popped at most once, and each iteration of the main loop writes at least one entry of POS, which is never written twice, so there are at most n iterations. Turning POS and LEN into factors is one more pass and encoding them one more, so the whole encoder runs in $\Theta(n)$.

Decoding is linear, same as in LZ77, $\Theta(n)$.

The memory is $\Theta(n)$, and here the constant matters. The algorithm holds S , SA, LCP, POS and LEN at the same time, which counts as $17n$ bytes for *CPS1a* ([13]).

Chapter 5

LZ77: Kärkkäinen–Kempa–Puglisi variant

5.1 Idea

The previous chapter computed the same factorization as this one, and by the same route up to a point: build SA, read the repetitions off it, turn them into factors. What [3] changes is the second step. CPS needs the LCP array to know how long each repetition is, and holding it alongside SA. KKP3 never builds LCP at all, well, instead it builds other structures: NSV and PSV of Definition 1.11.

The observation behind it is that the LCP array answers a question we do not quite need. It gives the agreement between every pair of neighbours in SA, while the factorization only needs, for each position i , the single “best” earlier occurrence. It turns out that this best occurrence can be found among just two candidates, and that the two can be computed from SA alone, without any lengths at all. It appears that the lengths can be then obtained by comparing symbols directly in linear time.

Of the algorithms proposed in [3] we implement the first one, which the authors call KKP3 after the three arrays it keeps.

5.1.1 KKP3 approach description

The output is the same as in Chapter 4: the arrays POS and LEN of Definition 1.6, from which the factors are formed by one left-to-right walk. Only the way they are obtained differs in a way, that only portion of positions are calculated.

Recall from Chapter 4 that suffixes sharing a long prefix stand close together in SA, and the further apart two suffixes are in SA, the shorter their common prefix. For a position i we are looking for an earlier position, that is, a suffix $j < i$ agreeing with suffix i on their prefixes as long as possible. By the property just recalled, the best such j must be the one lying closest to i in SA among all suffixes with a smaller text position. There are exactly two of those, one on each side, and they are exactly PSV[i] and NSV[i] from Definition 1.11. This gives the lemma the whole algorithm rests on, and it is proved in correctness part.

Lemma 5.1 (Two candidates suffice, [3]). *For every position i ,*

$$\text{LEN}[i] = \max\{\text{lcp}(i, \text{PSV}[i]), \text{lcp}(i, \text{NSV}[i])\},$$

and $\text{POS}[i]$ may be taken to be whichever of the two attains the maximum.

So the algorithm splits in two. First compute PSV and NSV, which only needs SA. Then, at each position where a factor begins, compare symbols against the two candidates and keep the one with larger common prefix.

It remains to compute the two arrays. Both are obtained in a single left-to-right pass over SA with a stack holding those entries that have not yet met a smaller value. When the entry $\text{SA}[i]$ is read, every stacked value greater than it is popped, and each popped value has just found its next smaller value, namely $\text{SA}[i]$, while its previous smaller value is the entry sitting below it on the stack. The value $\text{SA}[i]$ is then pushed. Two sentinel zeros, one before the array and one after, make sure that every entry is eventually popped.

5.1.2 Pseudocode

In our implementation the stack is kept inside SA itself: the cells $\text{SA}[0 \dots \text{top}]$ hold the stack, with $\text{SA}[0] = 0$ acting as the sentinel (Definition 1.3) at the bottom, while $\text{SA}[i]$ for $i > \text{top}$ is still the unmodified suffix array. This is possible because $\text{top} \leq i$ at every step, so a cell is overwritten only after it has been read. The stack needs no memory of its own: it is kept in already processed part of SA.

Algorithm 7 KKP3: computing PSV, NSV and the factors

```
1: procedure COMPUTESMALLERVALUES(SA,  $n$ )
2:   SA[0]  $\leftarrow$  0,   SA[ $n + 1$ ]  $\leftarrow$  0            $\triangleright$  sentinels at both ends
3:    $top \leftarrow 0$ 
4:   for  $i \leftarrow 1$  to  $n + 1$  do
5:     while SA[ $top$ ] > SA[ $i$ ] do
6:       NSV[SA[ $top$ ]]  $\leftarrow$  SA[ $i$ ]
7:       PSV[SA[ $top$ ]]  $\leftarrow$  SA[ $top - 1$ ]
8:        $top \leftarrow top - 1$ 
9:        $top \leftarrow top + 1$ ,   SA[ $top$ ]  $\leftarrow$  SA[ $i$ ]    $\triangleright$  push, overwriting a cell already processed

10: function LCP( $S$ ,  $n$ ,  $i$ ,  $j$ )
11:   if  $j = 0$  then
12:     return 0            $\triangleright$  no candidate on that side
13:    $l \leftarrow 0$ 
14:   while  $i + l \leq n$  and  $j + l \leq n$  and  $S[i + l] = S[j + l]$  do
15:      $l \leftarrow l + 1$ 
16:   return  $l$ 

17: function FACTORIZE( $S$ ,  $n$ , PSV, NSV)
18:    $triples \leftarrow$  empty list,    $i \leftarrow 1$ 
19:   while  $i \leq n$  do
20:      $l_p \leftarrow$  LCP( $S$ ,  $n$ ,  $i$ , PSV[ $i$ ])
21:      $l_n \leftarrow$  LCP( $S$ ,  $n$ ,  $i$ , NSV[ $i$ ])
22:     if  $l_p > l_n$  then
23:        $(p, l) \leftarrow$  (PSV[ $i$ ],  $l_p$ )
24:     else
25:        $(p, l) \leftarrow$  (NSV[ $i$ ],  $l_n$ )
26:     if  $i + l > n$  then
27:        $l \leftarrow n - i$             $\triangleright$  leaving one position for LZ77 mismatch symbol
28:     if  $l > 0$  then
29:        $triples.append(p, l, S[i + l])$ 
30:     else
31:        $triples.append(1, 0, S[i])$     $\triangleright$  no earlier occurrence, the pointer is unused
32:      $i \leftarrow i + l + 1$ 
33:   return  $triples$ 
```

The encoder and the decoder are the same as in Chapter 4: the factors are triples, the codewords have the fixed length $L_c = L_p + L_l + 1$ with $L_p = \lceil \log_\alpha |S| \rceil$, and the decoder copies symbol by symbol from the output built so far in order to enable overlapping.

5.2 Correctness

Throughout this section we denote by S the source, $n = |S|$, and SA the suffix array of S (see Definition 1.7). We first prove Lemma 5.1, then the claim that the scan computes PSV

and NSV correctly.

5.2.1 Two candidates suffice

Proof of Lemma 5.1. $\text{ISA}[i]$ is the position of suffix i in SA (Definition 1.10). For any two suffixes, the length of their common prefix is the smallest LCP value lying between them in SA, it means that $\text{lcp}(i, j)$ can only decrease as $\text{ISA}[j]$ moves away from $\text{ISA}[i]$ in either direction.

Let $j < i$ be the earlier position achieving the longest common prefix with suffix i . Suppose $\text{ISA}[j]$ lies to the left of $\text{ISA}[i]$ and that $j \neq \text{PSV}[i]$. By Definition 1.11 the entry $\text{PSV}[i]$ is then strictly between them in SA, and it satisfies $\text{PSV}[i] < i$, so it is also an earlier position. By the property above, $\text{lcp}(i, \text{PSV}[i]) \geq \text{lcp}(i, j)$, so $\text{PSV}[i]$ is at least as good and the maximum is attained there. The case where $\text{ISA}[j]$ lies to the right is symmetric and gives $\text{NSV}[i]$.

If no earlier position exists at all, both candidates are equal to 0 and LCP returns 0 for each of them, which adheres to the rules of Definition 1.5. $\square$

The scan computes the two arrays

Lemma 5.2 (Stack invariant). *At the start of every iteration of the outer loop of COMPUTESMALLERVALUES in Algorithm 7, the cells $\text{SA}[0 \dots \text{top}]$ hold*

$$\text{SA}[0] = 0 < \text{SA}[1] < \dots < \text{SA}[\text{top}],$$

they are exactly the entries read so far that have not yet been followed by a smaller one, and $\text{top} \leq i$.

Proof. Initially $\text{top} = 0$ and the stack holds only the sentinel $\text{SA}[0] = 0$, which is smaller than every text position and therefore never popped, so the invariant holds.

At each iteration the inner loop pops exactly those entries greater than $\text{SA}[i]$. Since the stack increases from bottom to top, these form a contiguous block at the top, and the loop stops at the first entry smaller than $\text{SA}[i]$. Pushing $\text{SA}[i]$ afterwards therefore keeps the stack increasing.

And the last, top grows by one per iteration at most, while i grows by one always, so $\text{top} \leq i$ is preserved and the cell of SA written on a push has already been read in an earlier iteration. $\square$

Theorem 5.3 (The scan is correct). *After COMPUTESMALLERVALUES has finished, PSV and NSV hold the values of Definition 1.11 for every position of S .*

Proof. The moment an entry p is popped while $\text{SA}[i]$ is being read. Everything read between the push of p and this moment was greater than p , since otherwise p would have been popped earlier, then $\text{SA}[i]$ is the first entry after p in SA that is smaller, which is exactly $\text{NSV}[p]$. By Lemma 5.2 the entry sitting below p is the last one read before p that is smaller than it, everything between them having already been popped, and that is exactly $\text{PSV}[p]$.

Every entry is eventually popped: the sentinel $\text{SA}[n+1] = 0$ is smaller than every text position, so the last iteration empties the stack down to $\text{SA}[0]$. An entry that meets no smaller value on one side receives 0 there, what still fits Definition 1.11. $\square$

After computing PSV and NSV, FACTORIZE takes at position i the better of the two candidates, which by Lemma 5.1 gives the best factor. We always cover at least one symbol in a new triple on each step, and if $l_i \neq 0$ covers $l_i + 1$ of source, so this process terminates on approaching $|S|$.

The decoder is the same as in Chapter 4 and parses the same triples.

5.3 Complexity

Let $n = |S|$. Building SA by induced sorting ([16]), as already was mentioned in previous chapters, takes $\Theta(n)$ time.

The scan is linear. Every entry of SA is pushed exactly once and popped at most once by the stack, so the total number of stack operations is $\mathcal{O}(n)$, and the outer loop runs $\mathcal{O}(n)$ times, which gives $\mathcal{O}(n)$ operation on COMPUTESMALLERVALUES procedure.

The symbol comparisons are linear as well. At a position i where a factor begins, LCP is called twice and costs at most $l_p + l_n + 2$ comparisons, where $l_p, l_n \leq l_i$, so the cost is at most $2l_i + 2$, as a reminder, l_i is the length of the found factor, and since in FACTORIZE we have $2 \cdot \sum_{i=1}^N (l_i + 1) = 2n$, where N is number of triples, we have total time of LCP computing as $\mathcal{O}(n)$.

Decoding is $\Theta(n)$, exactly as in Chapter 4.

The memory is $\Theta(n)$, and the constant is where KKP3 gains over the algorithm of Chapter 4, but only on one array of length n . It holds S , SA, PSV and NSV, and the stack is stored inside the copy of SA rather than separately.

Chapter 6

Comparison and Experiments

6.1 Comparison

6.1.1 Repository

All codes of algorithms live at:

<https://github.com/krzysztof-turowski/string-algorithms/pull/62>.

algorithm	source	reference
LZ77 sliding window	<code>compression/lz77_sliding_window.py</code>	[1]
LZ78 block	<code>compression/lz78_block.py</code>	[2]
LZ77 CPS1a	<code>compression/lz77_cps.py</code>	[13]
LZ77 KKP3	<code>compression/lz77_kkp3.py</code>	[3]

Table 6.1: Implementations compared in this chapter.

6.1.2 Approaches

As we already described, we implemented online versions of LZ77 and LZ78 and offline version CPS1a, but LZ78 can be treated as offline by setting the size of block to full length of text.

LZ77 and LZ78 are streaming algorithms with a working set independent of the source length, while CPS1a is not, and its memory grows with the source. They can be turned offline by setting their bound to the length of the source: the window of LZ77 ($n = 2L_s = 2|S|$) then covers the whole text and the dictionary of LZ78 ($n = |S|$) is never cleared.

LZ77, LZ77CPS1a, LZ77KKP3 produce output, which consists of triples of the form (position, length, mismatch symbol) and differ only in the range a position may take, which is why they share a decoder. LZ78 emits pairs of the form (phrase index, symbol) with no length at all, and its code-words are of varying width, which makes its output a bit stream rather than a sequence of symbols of $\mathcal{A}$.

6.1.3 Time and Space

Let $n = |S|$ for the length of the source. For LZ77, n is the buffer length and L_s the maximal word length. For LZ78, w is the block length. Then from previous chapters we have

	LZ77	LZ78	CPS1a	KKP3
Encoding time	$\mathcal{O}(w \cdot n)$	$\mathcal{O}(n)$ expected	$\Theta(n)$	$\Theta(n)$
Decoding time	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Working memory	$\Theta(w)$	$\mathcal{O}(w)$	$\Theta(n)$	$\Theta(n)$
online	yes	yes	no	no

Table 6.2: Time and space of presented algorithms.

CPS1a against KKP3

The two compute the same thing and differ only in how they find the lengths. Both start from SA, both produce POS and LEN, and both run in $\Theta(N)$.

CPS1a reads the lengths off the LCP array, which has to be built first by a full linear pass [15], and then walks SA once with a stack, filling an entry of POS at every step. KKP3 does not use LCP array directly, it walks SA once with a stack as well, but to obtain PSV and NSV, the lengths are then recovered separately by comparing symbols directly, but only a portion of positions, because it makes factorization of string on run, without preprocessing arrays of POS, LEN at all, because by Lemma 5.1 only two candidates ever have to be compared for any position, and by the analysis of Section 5.3 the comparisons cost $\mathcal{O}(N)$ in total because they are made only where a factor begins.

In terms of implementation both algorithms work on the same SA from [16], CPS1a pays for the LCP array and for the two output arrays, KKP3 for NSV and PSV alone, which is one integer array less and no separate stack.

6.2 Experiments

All data modules live at <https://github.com/krzysztof-turowski/string-algorithms>.

6.2.1 Runs

All texts come from the generators that ship with the `string-algorithms` repository, so that the experiments can be reproduced without any additional data. Four modules are used:

- `generator/named.py` for the two morphic words,
- `generator/rand.py`
- `generator/input/pantadeusz.txt` example of real text,
- `generator/exact_string_matching.py` for worst case.

text	description	generator	α
Fibonacci	the Fibonacci word	fibonacci_word	2
Thue-Morse	the Thue-Morse word	thue_word	2
random binary	uniform over two letters	random_word	2
random 26	uniform over the lowercase Latin alphabet	random_word	26
natural	Pan Tadeusz	natural_generator	33

Table 6.3: Texts used in the experiments.

Every text is truncated to the same length n , so that the rows of one table are directly comparable. Each algorithm runs in its *OFFLINE* configuration, with the whole text available from start.

Each table reports, for one text and all four implementations:

- n — length of source
- α — size of alphabet
- z — the number of codewords produced by algorithm
- ρ — the compression ratio, computed according to Definition 1.21.
- **encode** — working time of encoder in ms
- **decode** — working time of decoder in ms

Fibonacci

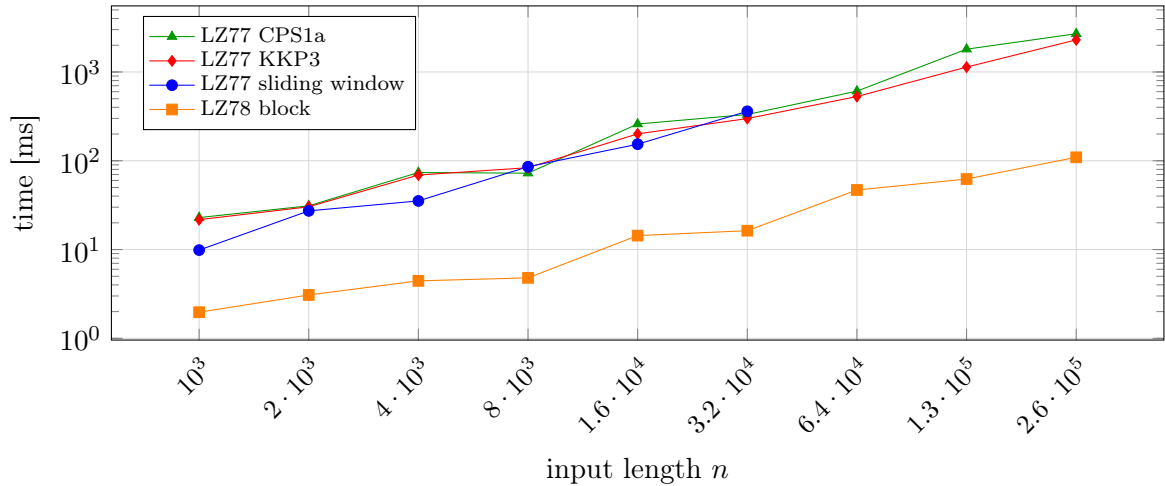


Figure 6.1: Fibonacci, encoding time.

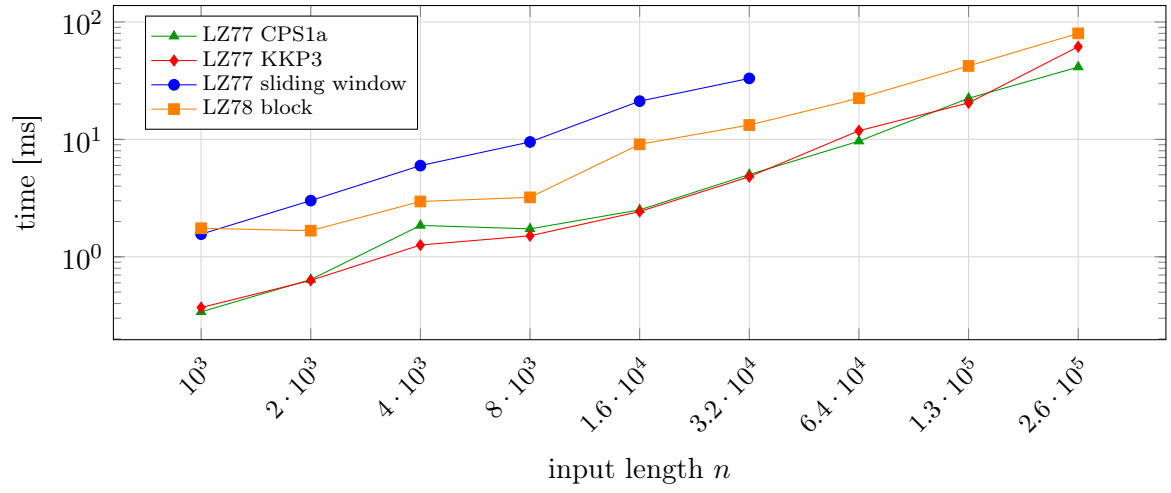


Figure 6.2: Fibonacci, decoding time.

algorithm	n	α	z	ρ	encode [ms]	decode [ms]
LZ77 sliding window	32000	2	22	0.0213	469.08	64.52
LZ78 block	32000	2	1050	0.3298	152.9	35.42
LZ77 CPS1a	32000	2	22	0.0206	605.42	8.11
LZ77 KKP3	32000	2	22	0.0206	545.25	10.96

Table 6.4: Fibonacci: the Fibonacci word.

Thue-Morse

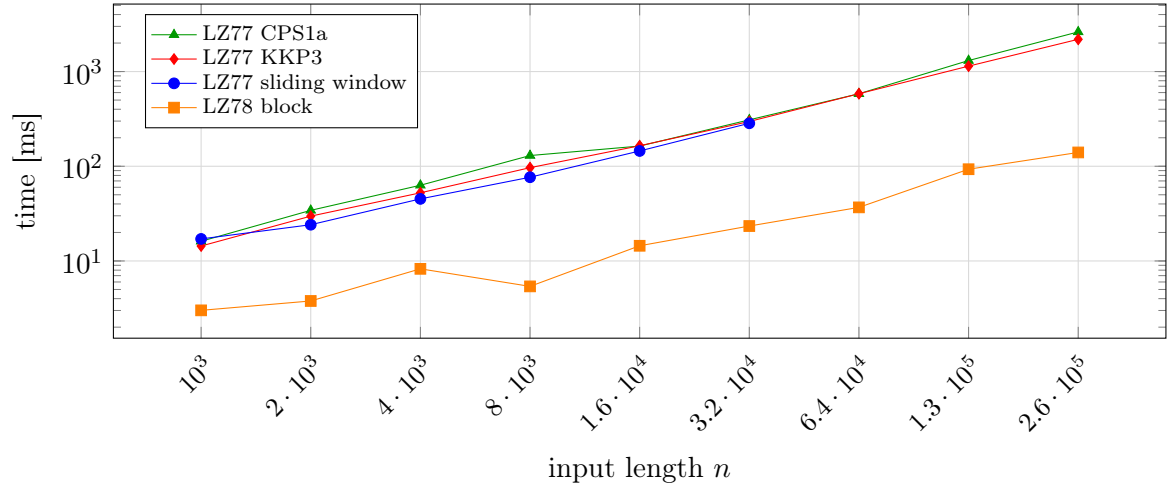


Figure 6.3: Thue-Morse, encoding time.

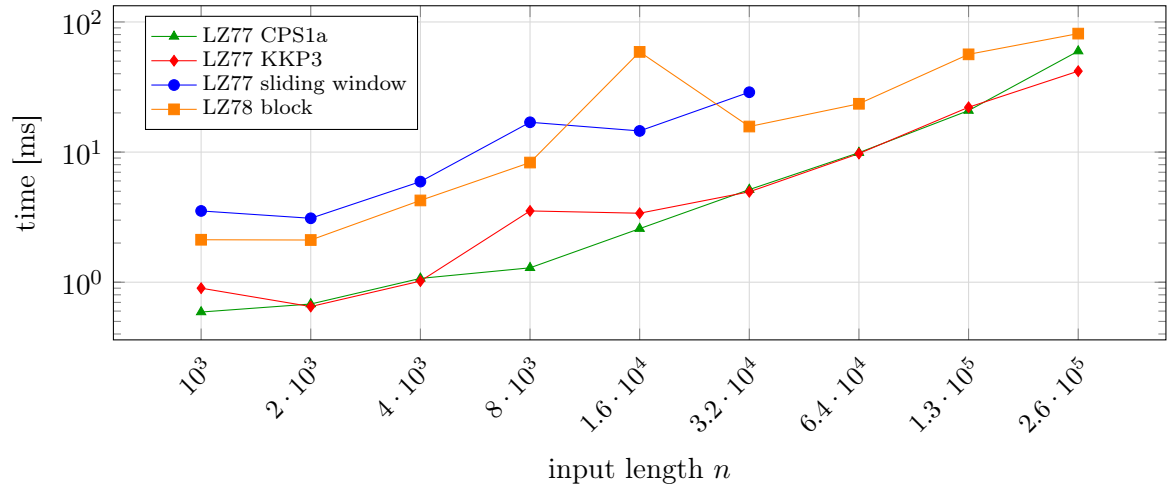


Figure 6.4: Thue-Morse, decoding time.

algorithm	n	α	z	ρ	encode [ms]	decode [ms]
LZ77 sliding window	32000	2	29	0.0281	461.67	49.65
LZ78 block	32000	2	1425	0.4704	29.4	26.28
LZ77 CPS1a	32000	2	29	0.0263	448.91	8.45
LZ77 KKP3	32000	2	29	0.0263	410.0	8.69

Table 6.5: Thue-Morse: the Thue-Morse word.

Random 26

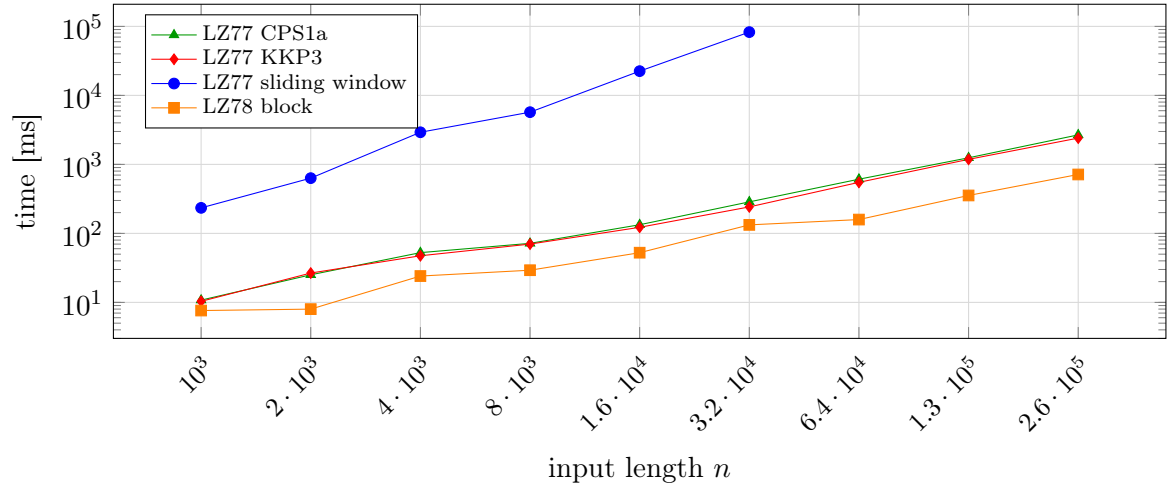


Figure 6.5: random 26, encoding time.

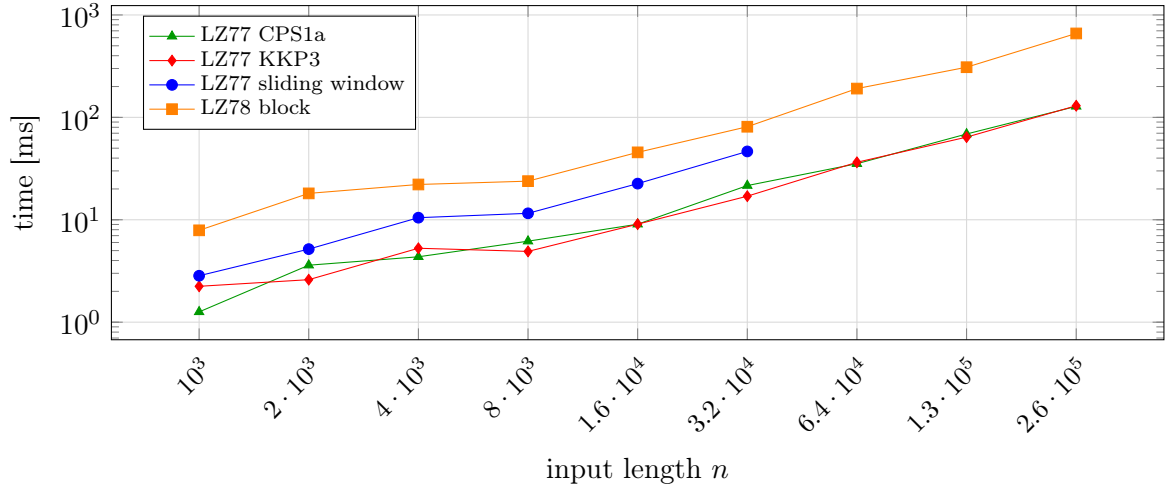


Figure 6.6: random 26, decoding time.

algorithm	n	α	z	ρ	encode [ms]	decode [ms]
LZ77 sliding window	32000	26	9083	2.5546	81989.29	51.75
LZ78 block	32000	26	10192	1.1534	78.4	131.06
LZ77 CPS1a	32000	26	9083	1.7031	343.46	24.88
LZ77 KKP3	32000	26	9083	1.7031	297.18	17.13

Table 6.6: random 26: uniform over the lowercase latin alphabet.

Random binary

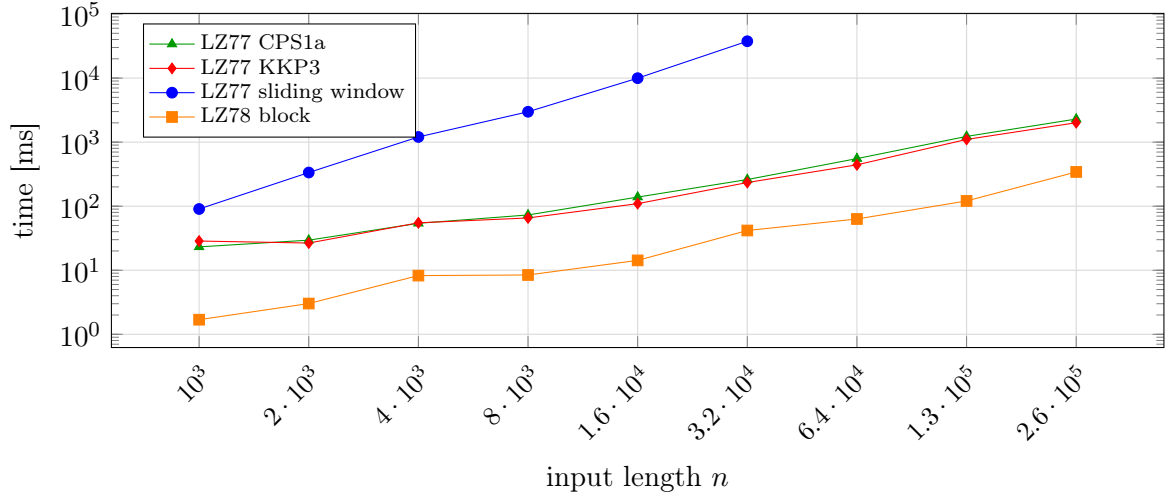


Figure 6.7: random binary, encoding time.

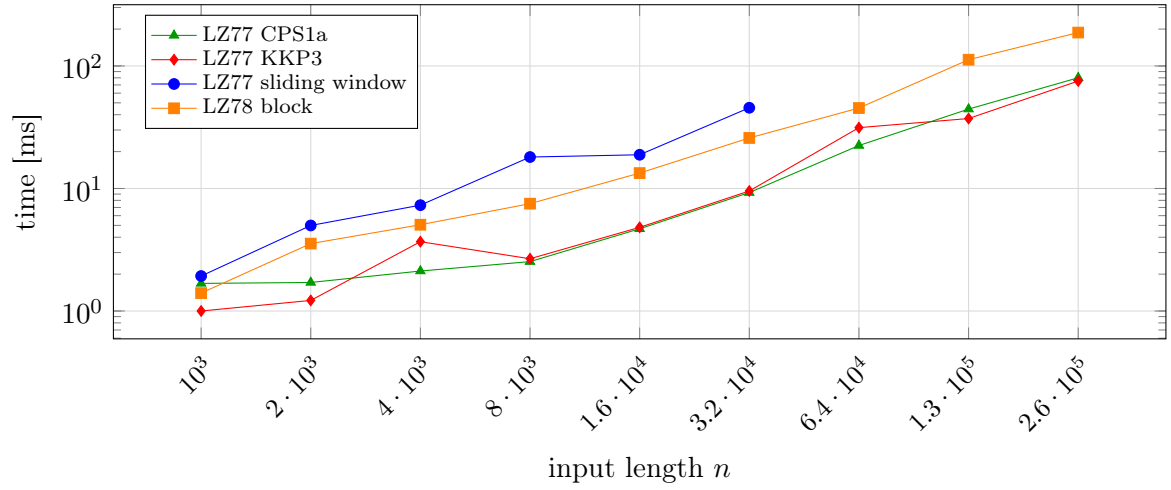


Figure 6.8: random binary, decoding time.

algorithm	n	α	z	ρ	encode [ms]	decode [ms]
LZ77 sliding window	32000	2	2175	2.107	37815.84	34.95
LZ78 block	32000	2	3218	1.1793	26.11	22.39
LZ77 CPS1a	32000	2	2175	1.4273	255.39	9.09
LZ77 KKP3	32000	2	2175	1.4273	220.57	10.32

Table 6.7: random binary: uniform over two letters.

Pan Tadeusz

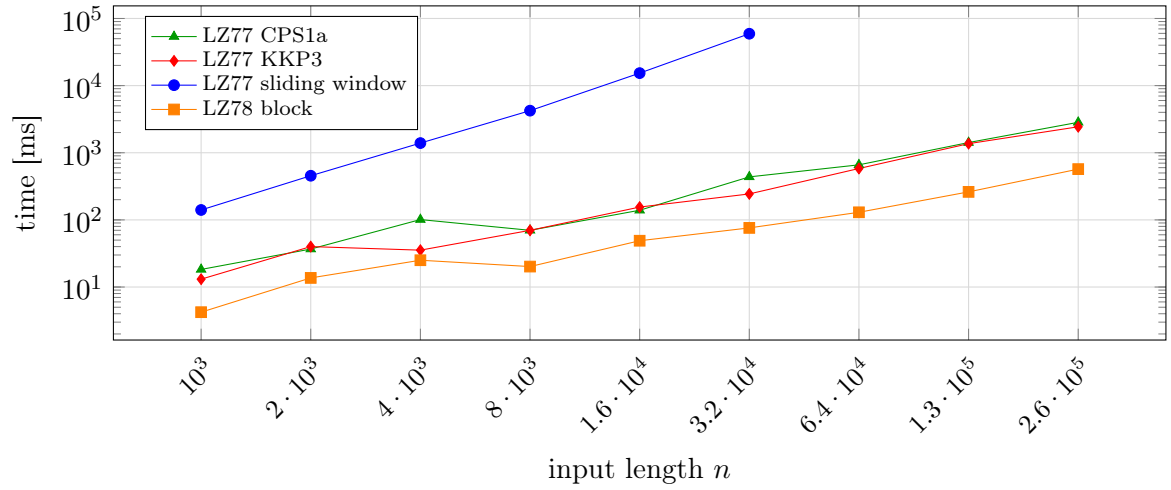


Figure 6.9: natural, encoding time.

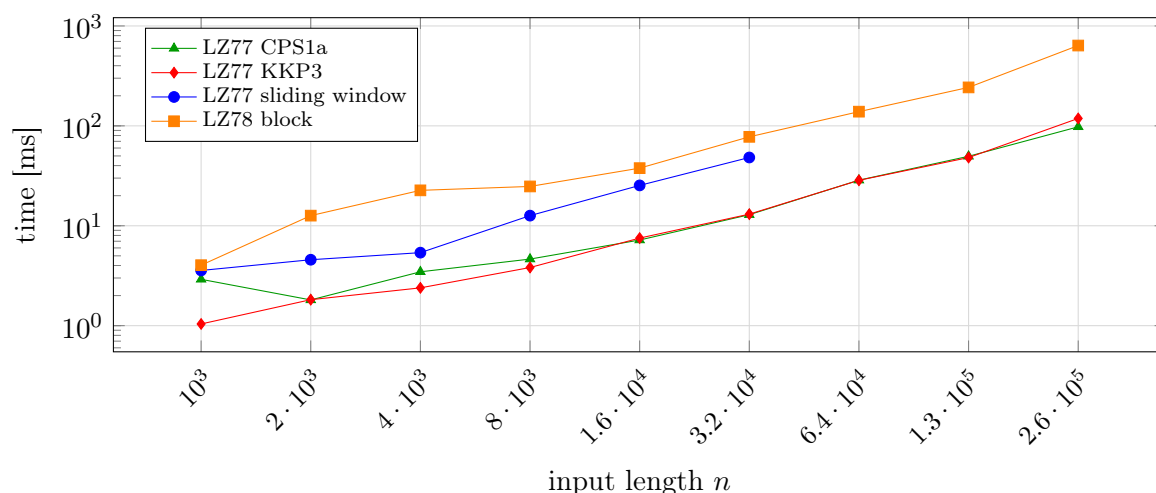


Figure 6.10: natural, decoding time.

algorithm	n	α	z	ρ	encode [ms]	decode [ms]
LZ77 sliding window	32000	34	6133	1.3416	55752.28	47.67
LZ78 block	32000	34	8078	0.8481	64.83	65.97
LZ77 CPS1a	32000	34	6133	0.9583	314.13	12.97
LZ77 KKP3	32000	34	6133	0.9583	281.1	25.07

Table 6.8: natural: Pan Tadeusz, letters and spaces only.

6.2.2 Results summary

The sliding-window LZ77 deserves a separate remark. Run offline, as all the experiments are, it has $L_s = |S|$, so its length field is as wide as the pointer field and every codeword carries $1 + 2\lceil \log_\alpha |S| \rceil$ symbols instead of $1 + \lceil \log_\alpha |S| \rceil + \lceil \log_\alpha (\max_i l_i + 1) \rceil$ of CPS1a and KKP3 which size the length field from the factors actually produced. The number of factors is identical — z agrees with CPS1a and KKP3 in every table, so the worse ratio is entirely an artifact of its implementation for rather online usage. What the tables really show about this implementation is its running time: since with the window equal to the whole text the naive search of REPRODUCIBLEEXTENSION (see Definition 1.15) becomes up to quadratic.

The number of factors and the compression ratio. On the compressible texts: Fibonacci, Thue-Morse and the run of one closed letter, the three LZ77 variants produce the same factorization and the same small number of factors, while LZ78 produces a lot more factors what is caused by *incremental parsing*, which doesn't support long overlapping factors. The gap is very big because of the property that LZ78 does not allow overlapping in the process of factor production. The three LZ77 implementations agree on z in every table, as they must, since they compute the same factorization by three different routes.

On the incompressible (have small average factor length) texts the picture reverses, the gradually increasing codeword length covers the loss in incremental parsing. Note that it is natural, that we get a ratio greater than 1, because in such cases we face the problem, that the length of codeword eats away any possible gain in encoded representation in both LZ77 and LZ78 approaches.

The encoding time. What the tables really show about the sliding-window implementation is its running time, and here it is worth being precise about when it is slow, because it is not slow everywhere. Its cost is not driven by the length of the source alone but by the type of text itself: on one type it can run in almost $\Theta(n)$ time, like in *Fibonacci* (see Table 6.4), on the other type it will cost $\Theta(n^{1.5})$ or more, like in *random-26* (see Table 6.6). Overall *CPS1a* and *KKP3* pay for not small amount of additional structures on all types of text, so their additional constant in complexity makes them slower than *LZ78*.

The decoding time. Decoding behaves as expected, *LZ78* runs slower due to the greater amount of factors. *CPS1a* and *KKP3* decode fastest everywhere although they share the decoder with the sliding window implementation and parse the same triples. The difference is that the latter reconstructs the output inside a ring buffer, so every symbol access requires a lot of jumps in memory, while the originally offline variants copy from the output built so far.

Implementation details In *CPS1a* we use SA (in *KKP3* too) and LCP of Definitions 1.7 and 1.8. Both are already available in the repository and we use them as they are, rather than reimplementing them: SA is built by induced sorting, the algorithm of [16], and LCP by the linear-time method of [15].

Chapter 7

Future implementations

As future implementations one can consider other algorithms presented in [13] and the better versions of *KKP3* in [3]. But for proper extension for LZ compression algorithms the work [5] can be considered a very good example of what can be done in this field, since in their survey one can find a broad list of approaches mentioned.

There we will stop after explaining some sense behind other versions of algorithms presented in [13] and [3], other than those we implemented here.

7.1 *CPS* algorithms

7.1.1 *CPS1b* and *CPS1c*

These two are not new algorithms but space optimizations of the same scan we do in *CPS1a*, so the factorization they compute is exactly the one of *CPS1a*.

CPS1b rests on the observation that none of the pointers i_1, i_2, i_3 ever reaches a position i of SA for which the entry $\text{POS}[\text{SA}[i]]$ has already been written, so the cells left behind by the scan are free and the pair $(\text{position}, \text{length})$ can be written into them instead of into separate arrays. The text position being assigned goes into $\text{SA}[i_2]$ and the pointer it receives goes into $\text{LCP}[i_2]$, where i_2 is set to i_1 at the beginning of the backtracking loop. The pairs end up scattered over SA and LCP, but their correct order can be restored by an in-place pass, which will at the end place desirable POS array inside SA. One integer array of length n is thus removed.

CPS1c drops LEN from the scan as well, so that only POS is computed and no storage is allocated for the lengths at all, which frees the space of LCP once the scan is over. The lengths are then recovered on demand: we do not calculate it at each position, but only at needed ones for factorization, so if source string is cut in N factors, then we were computing only N lengths, which sum to $\Theta(n)$ comparisons. These values can be now stored inside previously freed LCP. One more integer array of length n is thus removed.

7.1.2 *CPS2*

CPS2 still requires SA, but removes LCP from needed structures and instead makes use of a structure for range minimum queries (RMQ) built on SA. $\text{RMQ}(a, b)$ returns the index of the smallest value among $\text{SA}[a], \dots, \text{SA}[b]$ in constant time after a preprocessing, which takes $\mathcal{O}(n)$ time, and $\mathcal{O}(n)$ space.

We are not going to precompute POS or LEN like before, but make factorization of string on fly. The approach is based on the fact that similar prefixes of suffixes in SA lie together in a continuous interval, and as the length of the common prefix rises this interval is narrowing, and also we can utilize the order of some i -th position of suffixes in this interval in SA since they are in lexicographical order.

The algorithm keeps three things: the suffix array SA, the range minimum structure over it, and the current interval $SA[a \dots b]$, which holds exactly those suffixes that begin with the part of the factor confirmed so far, $S[i \dots j - 1]$. The invariant maintained throughout is that at least one suffix of the interval starts to the left of i , as long as it holds, the substring $S[i \dots j - 1]$ has been seen earlier and the factor can be extended further. One step of the extension consists of three operations.

- (1) **REFINE**: the next symbol $S[j]$ is appended to the pattern and the interval is narrowed to the suffixes beginning with $S[i \dots j]$. Two binary searches suffice, so the step costs $\mathcal{O}(\log n)$.
- (2) **RMQ**(a, b): the smallest value of SA in the narrowed interval is found.
- (3) The minimum is compared with i . If it is smaller, an occurrence to the left exists, so it is kept as the pointer, j is increased and the step repeats. If it is not, the only occurrence is the suffix i itself, the extension fails and the factor ended at the previous step.

The worst case time becomes $\mathcal{O}(n \log n)$, since we have at most $n - 1$ steps and each one costs $\mathcal{O}(\log n)$.

7.1.3 CPS3

This approach gives up the usage of structures we deal with in our work, so the decision is to leave to the reader to familiarize themselves with *CPS3* in the work [13].

7.2 KKP algorithms

Besides *KKP3*, which is the version implemented here, [3] describes other versions of same approach but with reduced working memory.

7.2.1 KKP2s

The idea of the algorithm is to keep one array fewer. The scan of SA computes the NSV values only, and the PSV value of a position is not stored at all but produced at the moment the factorization reaches that position. What makes this possible is the array Φ , where $\Phi[i] = SA[ISA[i] - 1]$ is the immediate lexicographical predecessor of the suffix i , implemented as a circular linked list.

Restrict that list to the suffixes starting at or before a position t and call it Φ_t . Passing from Φ_{t-1} to Φ_t is a single insertion, and the place where the suffix t goes is given by its lexicographical neighbours among the suffixes $1, \dots, t - 1$, which are exactly $PSV[t]$ and $NSV[t]$.

Since Φ points at the predecessor and nothing stands between the two neighbours before the insertion,

$$PSV[t] = \Phi_{t-1}[NSV[t]],$$

so one array serves both roles in turn. The scan fills it with the NSV values, and the factorization loop rebuilds it into Φ from left to right by these restricted Φ_t , so that at the step t the entry $\Phi[t]$ still holds NSV $[t]$, while the entry it points to, lying to the left of t , has already been rebuilt and therefore holds PSV $[t]$. Two assignments then insert t into the list, and they are made at every position, whether or not a factor begins there, since a list left unrepaired would give wrong predecessors later on.

7.2.2 *KKP2n*

The idea of this variant is to not modify the suffix array given as input, without increasing the working memory. This desire comes from a fact that in previous variants of algorithm we were using processed positions of SA as stack, which is undesirable when the suffix array is needed afterwards. What makes this possible is observation that PSV pointers can replace the stack.

For this the PSV value has to be available before the pop that uses it, so it is computed when pushing rather than when popping, and the rest of the algorithm is mirrored accordingly. The scan now produces PSV instead of NSV, and Φ is replaced by the inverse permutation Φ^{-1} . At the step t the entry $\Phi^{-1}[t]$ still holds PSV $[t]$, while the entry it points to has already been rebuilt and holds NSV $[t]$.

Bibliography

- [1] Jacob Ziv and Abraham Lempel. “A Universal Algorithm for Sequential Data Compression”. In: *IEEE Transactions on Information Theory* 23.3 (1977), pp. 337–343. DOI: 10.1109/TIT.1977.1055714.
- [2] Jacob Ziv and Abraham Lempel. “Compression of Individual Sequences via Variable-Rate Coding”. In: *IEEE Transactions on Information Theory* 24.5 (1978), pp. 530–536. DOI: 10.1109/TIT.1978.1055934.
- [3] Juha Kärkkäinen, Dominik Kempa, and Simon Puglisi. “Linear Time Lempel–Ziv Factorization: Simple, Fast, Small”. In: *Combinatorial Pattern Matching (CPM 2013)*. Ed. by Johannes Fischer and Peter Sanders. Vol. 7922. Lecture Notes in Computer Science. Springer, 2013, pp. 189–200. DOI: 10.1007/978-3-642-38905-4_19. arXiv: 1212.2952.
- [4] Johannes Fischer, Tomohiro I, Dominik Köppl, and Kunihiko Sadakane. “Lempel-Ziv Factorization Powered by Space Efficient Suffix Trees”. In: *Algorithmica* 80.7 (2018), pp. 2048–2081. DOI: 10.1007/s00453-017-0333-1.
- [5] Anisa Al-Hafeedh, Maxime Crochemore, Lucian Ilie, Evguenia Kopylova, William Smyth, German Tischler, and Munina Yusufu. “A Comparison of Index-Based Lempel-Ziv LZ77 Factorization Algorithms”. In: *ACM Computing Surveys* 45.1, 5 (2012), 5:1–5:17. DOI: 10.1145/2379776.2379781.
- [6] Alberto Policriti and Nicola Prezza. “LZ77 Computation Based on the Run-Length Encoded BWT”. In: *Algorithmica* 80.7 (2018), pp. 1986–2011. DOI: 10.1007/s00453-017-0327-z.
- [7] Sebastian Kreft and Gonzalo Navarro. “Self-Indexing Based on LZ77”. In: *Combinatorial Pattern Matching (CPM 2011)*. Ed. by Raffaele Giancarlo and Giovanni Manzini. Vol. 6661. Lecture Notes in Computer Science. Springer, 2011, pp. 41–54. DOI: 10.1007/978-3-642-21458-5_6.
- [8] Travis Gagie, Paweł Gawrychowski, Juha Kärkkäinen, Yakov Nekrich, and Simon Puglisi. “LZ77-Based Self-Indexing with Faster Pattern Matching”. In: *LATIN 2014: Theoretical Informatics*. Ed. by Alberto Pardo and Alfredo Viola. Vol. 8392. Lecture Notes in Computer Science. Springer, 2014, pp. 731–742. DOI: 10.1007/978-3-642-54423-1_63.
- [9] Wojciech Rytter. “Application of Lempel–Ziv Factorization to the Approximation of Grammar-Based Compression”. In: *Theoretical Computer Science* 302.1–3 (2003), pp. 211–222. DOI: 10.1016/S0304-3975(02)00777-6.
- [10] James Storer and Thomas Szymanski. “Data Compression via Textual Substitution”. In: *Journal of the ACM* 29.4 (1982), pp. 928–951. DOI: 10.1145/322344.322346.
- [11] Terry Welch. “A Technique for High-Performance Data Compression”. In: *Computer* 17.6 (1984), pp. 8–19. DOI: 10.1109/MC.1984.1659158.
- [12] Edward Fiala and Daniel Greene. “Data Compression with Finite Windows”. In: *Communications of the ACM* 32.4 (1989), pp. 490–505. DOI: 10.1145/63334.63341.

- [13] Gang Chen, Simon Puglisi, and William Smyth. “Lempel–Ziv Factorization Using Less Time & Space”. In: *Mathematics in Computer Science* 1.4 (2008), pp. 605–623. DOI: 10.1007/s11786-007-0024-4.
- [14] Yuto Nakashima, Tomohiro I, Shunsuke Inenaga, Hideo Bannai, and Masayuki Takeda. “Constructing LZ78 Tries and Position Heaps in Linear Time for Large Alphabets”. In: *Information Processing Letters* 115.9 (2015), pp. 655–659. DOI: 10.1016/j.ip1.2015.04.002.
- [15] Toru Kasai, Gunho Lee, Hiroki Arimura, Setsuo Arikawa, and Kunsoo Park. “Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications”. In: *Combinatorial Pattern Matching (CPM 2001)*. Ed. by Amihood Amir and Gad Landau. Vol. 2089. Lecture Notes in Computer Science. Springer, 2001, pp. 181–192. DOI: 10.1007/3-540-48194-X_17.
- [16] Ge Nong, Sen Zhang, and Wai Hong Chan. “Linear Suffix Array Construction by Almost Pure Induced-Sorting”. In: *2009 Data Compression Conference (DCC 2009)*. Ed. by James Storer and Michael Marcellin. IEEE Computer Society, 2009, pp. 193–202. DOI: 10.1109/DCC.2009.42.
- [17] Richard Karp, Raymond Miller, and Arnold Rosenberg. “Rapid Identification of Repeated Patterns in Strings, Trees and Arrays”. In: *Proceedings of the 4th Annual ACM Symposium on Theory of Computing (STOC 1972)*. ACM, 1972, pp. 125–136. DOI: 10.1145/800152.804905.
- [18] Juha Kärkkäinen and Peter Sanders. “Simple Linear Work Suffix Array Construction”. In: *Automata, Languages and Programming (ICALP 2003)*. Ed. by Jos Baeten, Jan Karel Lenstra, Joachim Parrow, and Gerhard Woeginger. Vol. 2719. Lecture Notes in Computer Science. Springer, 2003, pp. 943–955. DOI: 10.1007/3-540-45061-0_73.
- [19] Pang Ko and Srinivas Aluru. “Space Efficient Linear Time Construction of Suffix Arrays”. In: *Combinatorial Pattern Matching (CPM 2003)*. Ed. by Ricardo Baeza-Yates, Edgar Chávez, and Maxime Crochemore. Vol. 2676. Lecture Notes in Computer Science. Springer, 2003, pp. 200–210. DOI: 10.1007/3-540-44888-8_15.
- [20] N. Jesper Larsson and Kunihiro Sadakane. “Faster Suffix Sorting”. In: *Theoretical Computer Science* 387.3 (2007), pp. 258–272. DOI: 10.1016/j.tcs.2007.07.017.
- [21] Krzysztof Turowski et al. *string-algorithms: Collection of String Algorithms*. Software repository. 2020. URL: <https://github.com/krzysztof-turowski/string-algorithms> (visited on 09/25/2026).